Practical no. 1

Aim:- Creation of Test Suite Using Selenium IDE Tool

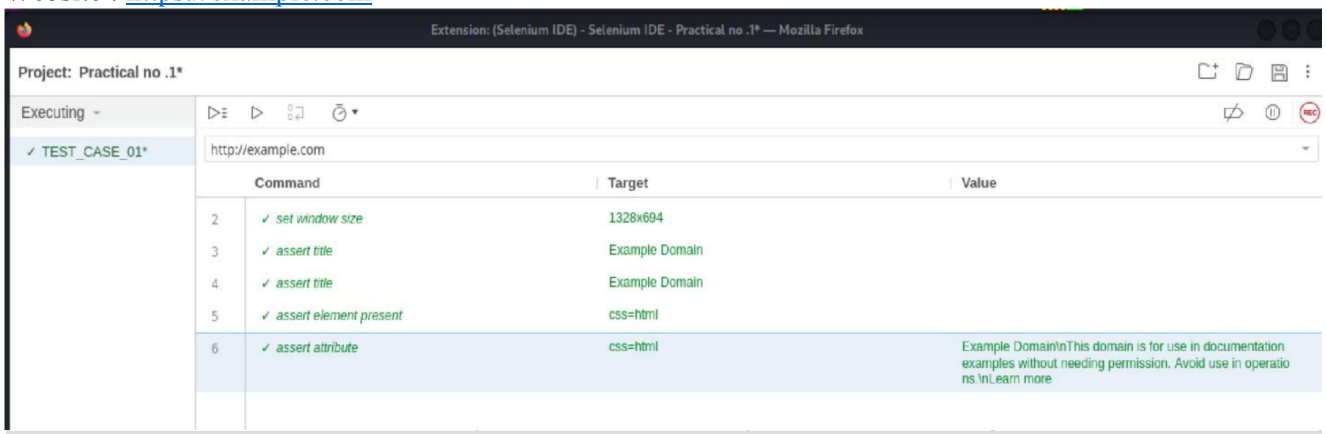
Requirements:-

1. **Selenium IDE**
2. **Browser with Extension (Chrome or Firefox)**

Test Case 1:-

HTML Page Validation :-

Website : https://example.com



Command	Target	Value	Purpose/Asser tion
open	[https://exa mple.com]	(https://ex ample.co m)	Loads the target HTML webpage.
setWindowSize	1328x694		Sets a uniform browser viewpoint size.
assertTitle	Example Domain		Verifies the <title> tag in the HTML <head> tag.

assertElementPresent	css=html		Ensures the main heading tag exists in the DOM.
asssertText	css=html	Example Domain	Verifies the text content inside the <h1> tag.
assertAttribute	css=a@href	[https://www.iana.org/domains/example	

Test Case 2:

XML Validation

Website : https://www.w3chool.com/xml/xml_examples.asp



Commands	Target	Val ue	Purpose/Asserti on
Open	https://www.w3schools.com/xml/xml_example s.asp		Opens XML documentation / example page
setWindowSize	1280x684		Sets viewport size
assertElementPresent	css=.withbookmark		verifies heading loads

assertText	css=.withbookmark	XML Exa mpl es	validates XML page title text
Close	alt+f4		closing the page

Test Case 3:

JSON API Output Validation

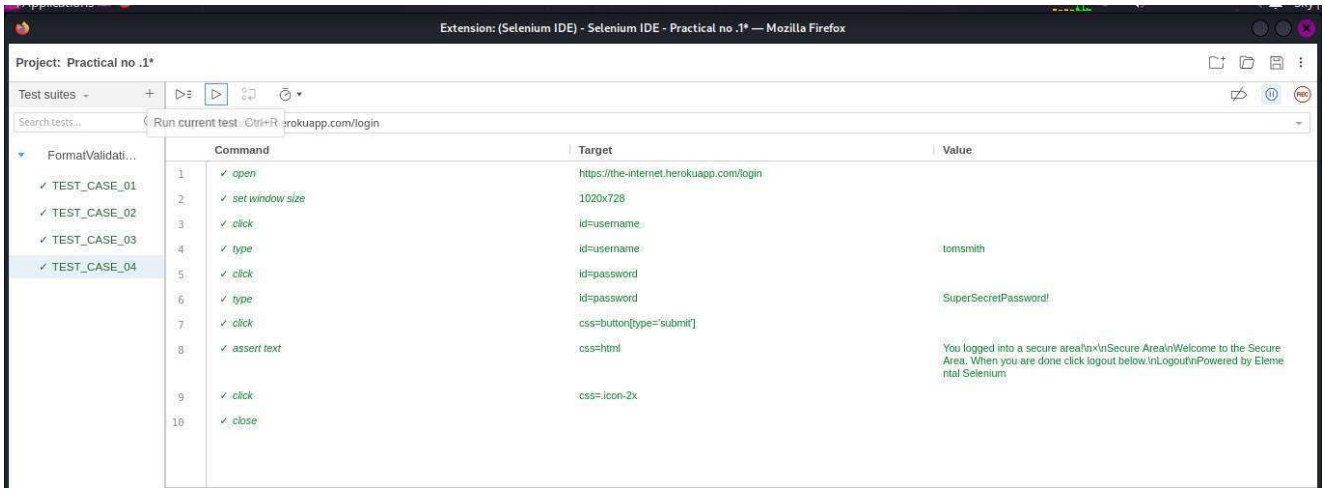
Website: https://jsonhero.io/



Commands	**Target**	**Value**	**Purpose/Asser tion**
open	https://jsonher o.io	https://jsonher o.io	opening the website.
setWindowSiz e	1280x800		windows size
assertElement Present	css=.itemsstretch		
assertText	css=.itemsstretch		
assertText	css=.itemsstretch		
close			closing the webpage

Test Case 4:

Dynamic form Validation



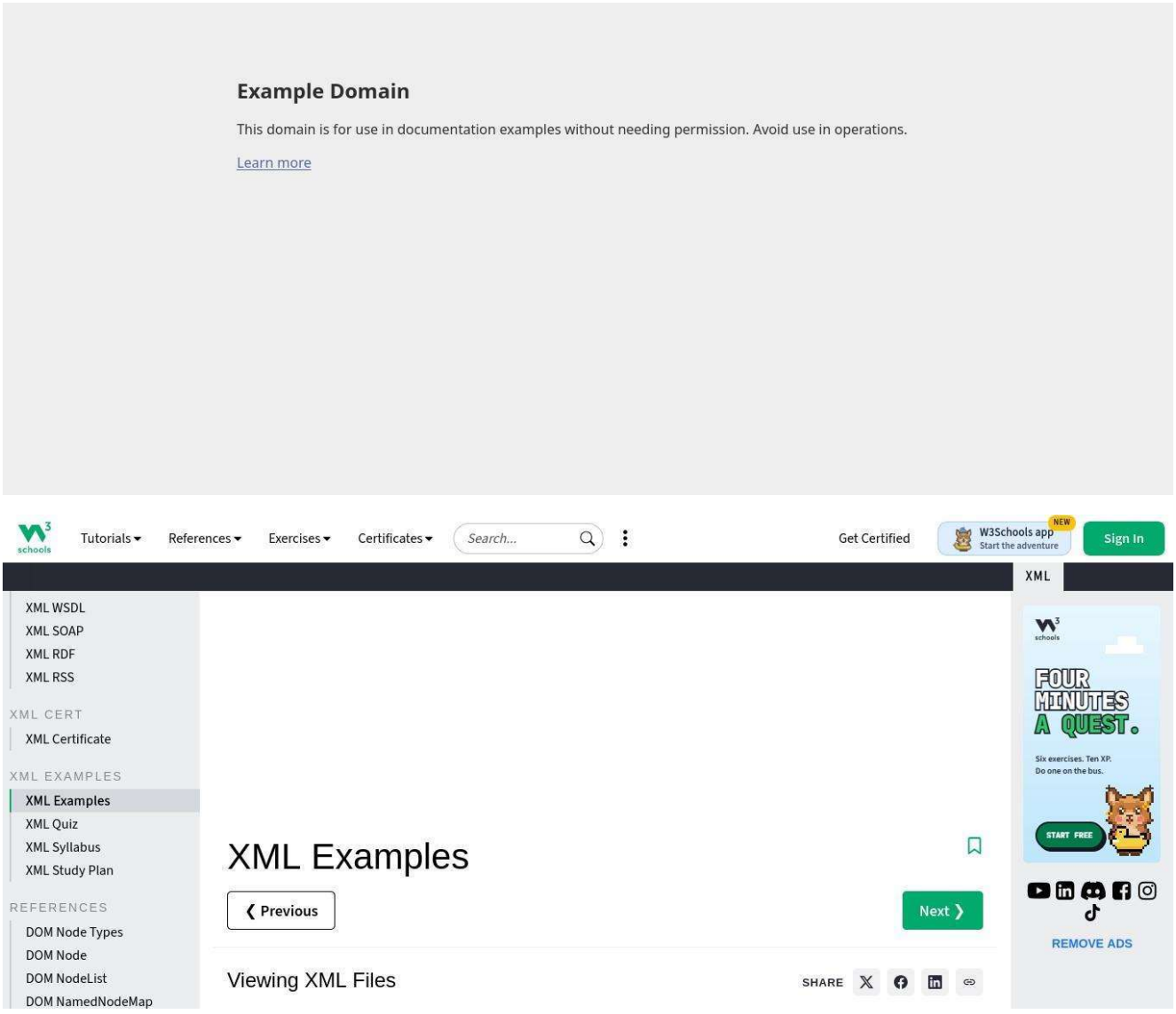
Commands	Target	Value	Purpose/Assertion
open	https://theinternet.herokuapp.com/l ogin		opening the webpage.
setWindow Size	1020x800		settingt the windows size.
click	id=username		click on the userid's textbox.
type	id=username	tomsmith	typing the text.
click	id=password		click on the passsword textbox.
type	id=password	SuperSecret Password!	typing the password.
click	css=button[t ype='submit']		button of login.
assert text	css=html	your are logging into secrue	flashing the text of logging.

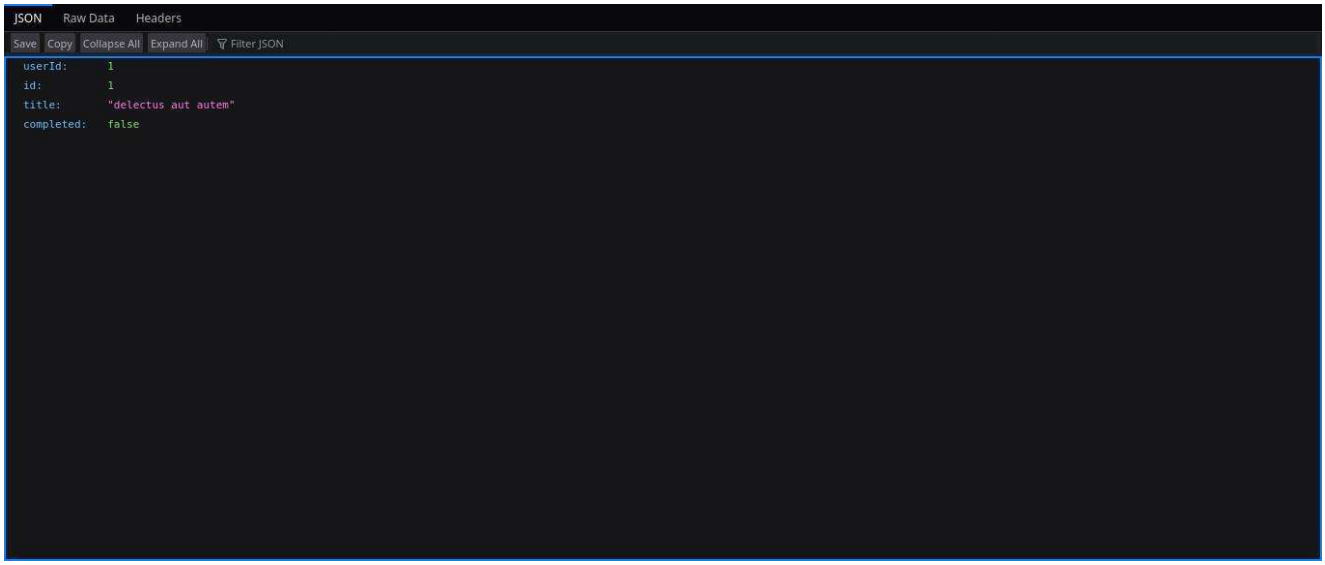
click	css=.icon-2x		logout buttton.
close			closing the webpage.

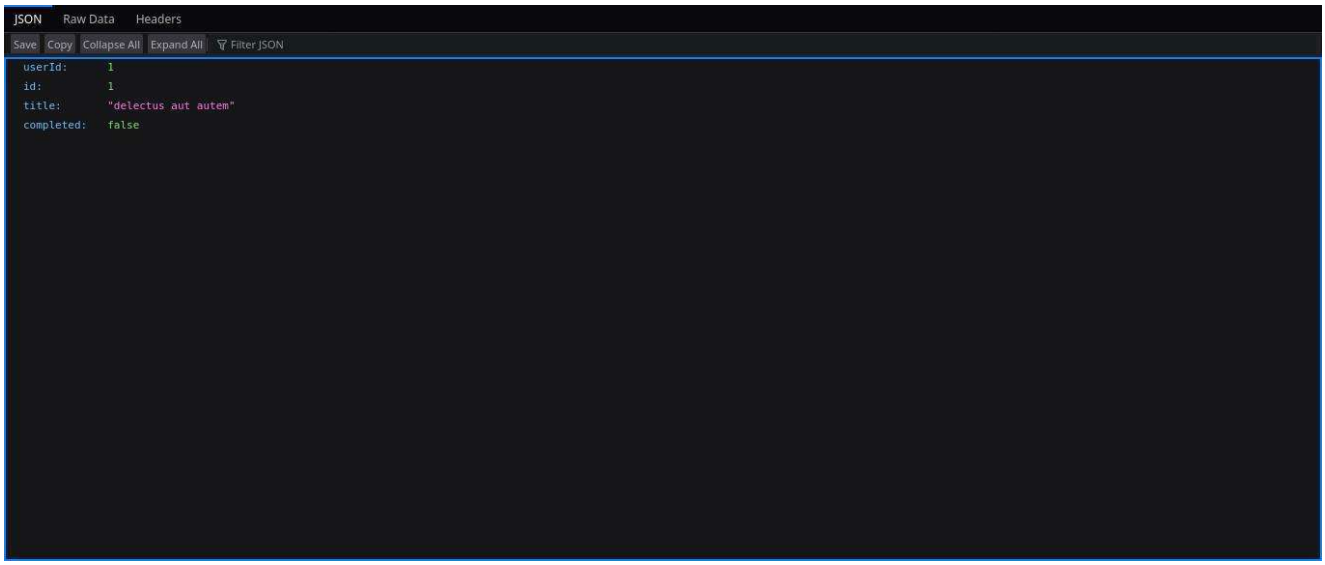
Test Case 5:

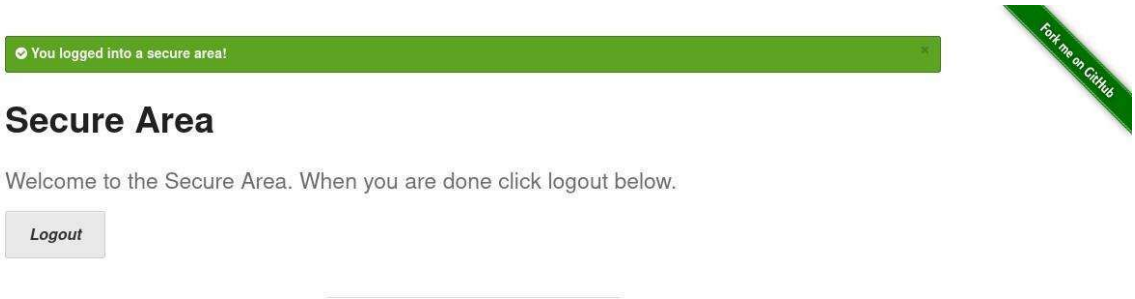
PyTest on Python

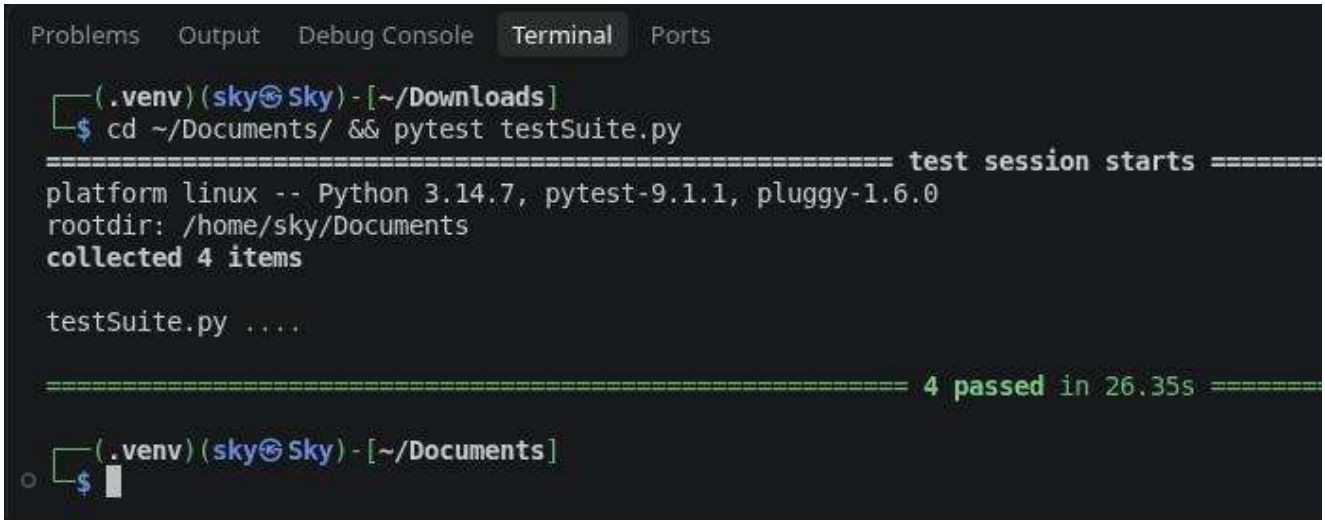
Execute the suit and export the automatation script in WebDrive Format.




JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
userId: 1
id: 1
title: "delectus aut autem"
completed: false


JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
userId: 1
id: 1
title: "delectus aut autem"
completed: false


You logged into a secure area!
Fork me on GitHub
Secure Area
Welcome to the Secure Area. When you are done click logout below.
Logout
Powered by Elemental Selenium



python code:

```
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestSuiteAllFormats:
    def setup_method(self, method):
        self.driver = webdriver.Firefox()
        self.driver.implicitly_wait(10)
        self.driver.maximize_window()

    def teardown_method(self, method):
        self.driver.quit()

    # -==============- TEST CASE 1: HTML VALIDATION -==============-

    def test_tc01_html_validation(self):
        self.driver.get("https://example.com/")

        # Verify page title and header text
        assert self.driver.title == "Example Domain"
```

```
    h1_element = self.driver.find_element(By.TAG_NAME, "h1")
    assert h1_element.text == "Example Domain"

# -==============- TEST CASE 2: XML VALIDATION -==============-

def test_tc02_xml_validation(self):
    self.driver.get("https://www.w3schools.com/xml/xml_examples.asp")
    # FIX: Explicitly wait for the main content h1 heading to be visible,
    # narrowing down the selector to the content container to skip hidden navigation tags.
    heading = WebDriverWait(self.driver, 10).until(
        EC.visibility_of_element_located((By.CSS_SELECTOR, "#main h1"))
    )
    assert 'XML Examples' in heading.text

# -==============- TEST CASE 3: JSON API Output VALIDATION -==============-

def test_tc03_json_validation(self):
    self.driver.get("https://jsonplaceholder.typicode.com/todos/1")
    # Locate text using wildcard XPath matching across raw browser viewport
    json_title = self.driver.find_elements(
        By.XPATH, "//*[contains(text(), 'delectus aut autem')]"
    )
    json_status = self.driver.find_elements(
        By.XPATH, "//*[contains(text(), 'completed')]"
    )
    assert len(json_title) > 0, "JSON key 'title' value missing from page output."
    assert len(json_status) > 0, "JSON key 'completed' flag missing from page output."

# -==============- TEST CASE 4: DYNAMIC FORM VALIDATION -==============-

def test_tc04_dynamic_form_validation(self):
    self.driver.get("https://the-internet.herokuapp.com/login")
```

```
# Interact with form controls
self.driver.find_element(By.ID, "username").send_keys("tomsmith")
self.driver.find_element(By.ID, "password").send_keys("SuperSecretPassword!")
self.driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
# Validate dynamic flash login response banner
flash_message = self.driver.find_element(By.ID, "flash")
assert "You logged into a secure area!" in flash_message.text
```

Practical No. 2

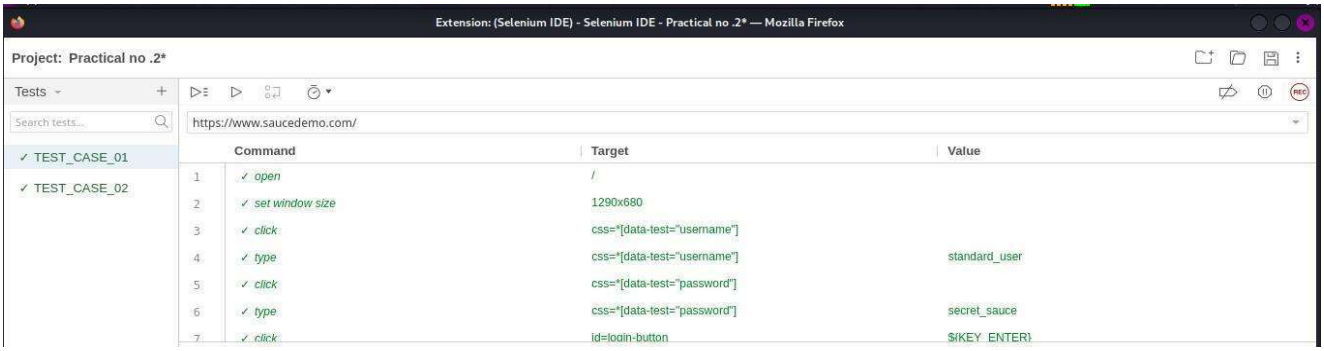
Aim: Cross-website Functional Testing Using Selenium IDE

Tools:

1. **Selenium IDE Develop Tools.**
2. **Testsuite for two Different Website.**

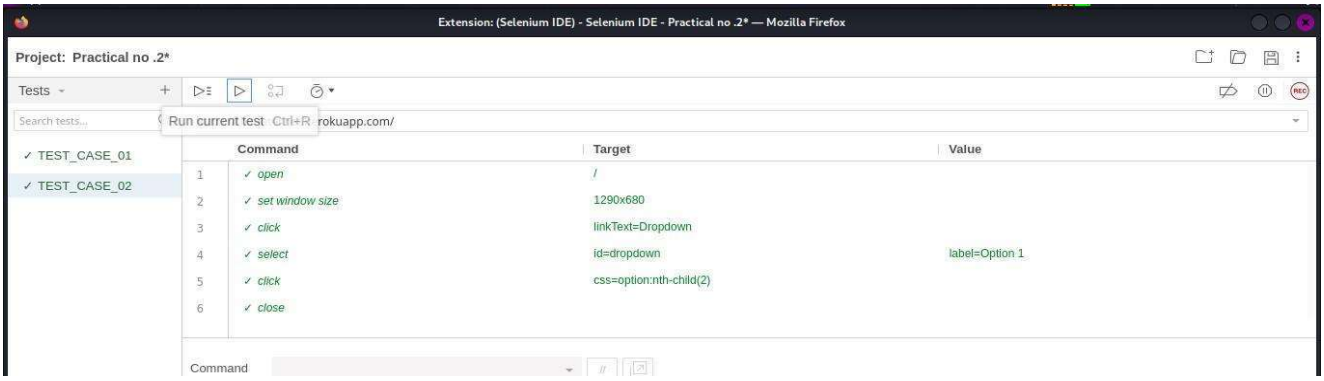
Test Case 1:

Website : https://saucedemo.com/



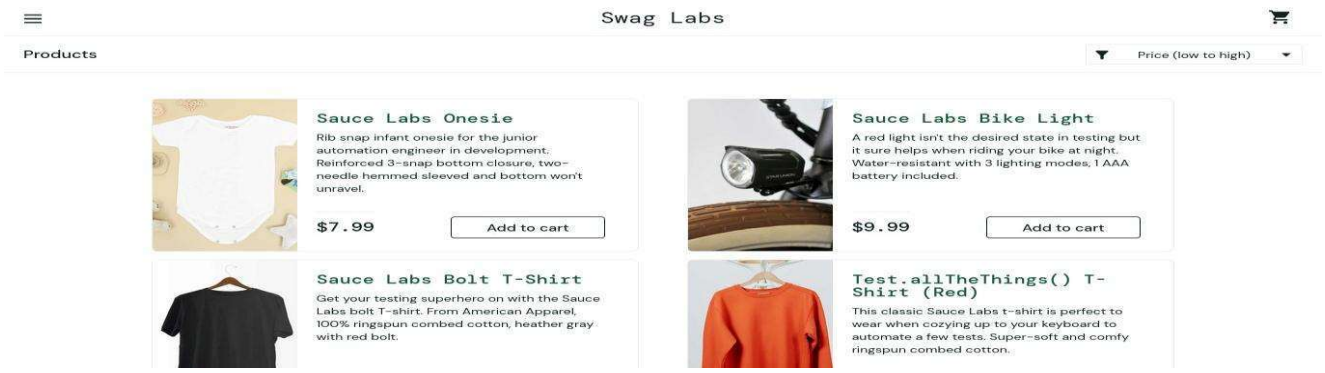
Test Case 2:

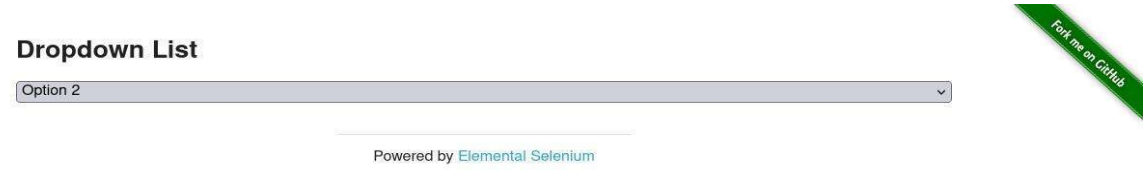
Website : https://th-internet-herokuapp.com/

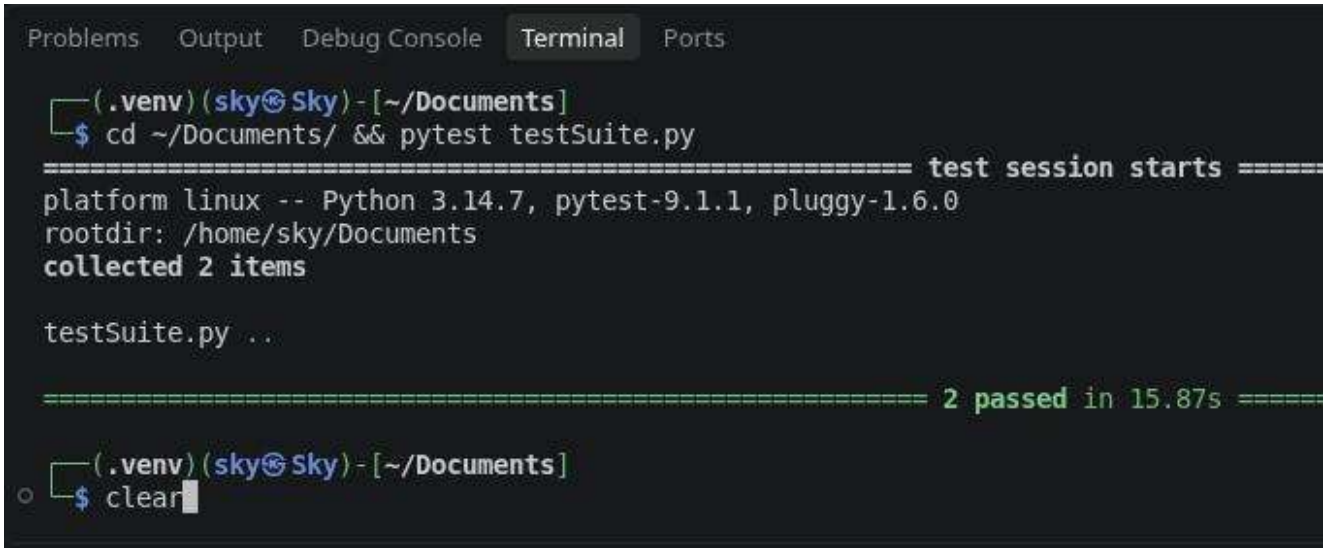


Test Case 3:

Test case using pytest to python







Python Code :

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
```

```
class TestCrossWebsiteFunctional:

    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.implicitly_wait(10)
        self.driver.maximize_window()

    def teardown_method(self, method):
        self.driver.quit()

    def test_tc01_saucedemo_functional(self):
        # 1. Open Website 1
        self.driver.get("https://www.saucedemo.com/")

        # 2. Form filling & submission
        self.driver.find_element(By.ID, "username").send_keys("standard_user")
        self.driver.find_element(By.ID, "password").send_keys("secret_sauce")
        self.driver.find_element(By.ID, "login-button").click()

        # 3. Content Assertion
        page_title = self.driver.find_element(By.CSS_SELECTOR, ".title").text
        assert page_title == "Products"

        # 4. Dropdown option selection
        sort_dropdown = Select(
            self.driver.find_element(
                By.CSS_SELECTOR, "[datatest='product-sort-container']"
            )
        )
        sort_dropdown.select_by_visible_text("Price (low to high)")

        # 5. Verify sorted content
```

```
    first_price = self.driver.find_element(
        By.CSS_SELECTOR, ".inventory_item_price"
    ).text
    assert first_price == "$7.99"

def test_tc02_herokuapp_functional(self):
    # 1. Open Website 2
    self.driver.get("https://the-internet.herokuapp.com/")

    # 2. Link Navigation
    self.driver.find_element(By.LINK_TEXT, "Dropdown").click()

    # 3. Content Assertion
    heading = self.driver.find_element(By.XPATH, "//h3").text
    assert heading == "Dropdown List"

    # 4. Dropdown Selection
    dropdown_element = Select(
        self.driver.find_element(By.ID, "dropdown")
    )
    dropdown_element.select_by_visible_text("Option 2")

    # 5. Verify Selected Option
    selected_option = dropdown_element.first_selected_option.text
    assert selected_option == "Option 2"
```

Practical No. 3

Aim: To design a Test Plan, write Test Scenarios and Test Cases for a given Web Application execute the test cases manually, and generate a Test Execution Report and Defect Report according to the Software Testing Life Cycle (STLC) process using Microsoft Excel / Google Sheets. Test Scenario : Check Login Functionality.

Steps:

- Understand Requirements: Identify the target module (e.g., User Login Page).
- Identify Test Scenarios: List all high-level testing possibilities (Positive, Negative, Boundary cases).
- Draft Test Cases in Excel: Create structured tables with standard columns:
- Test Case ID, Scenario Description, Pre-conditions, Test Steps, Test Data, Expected Result, Actual Result, Status (Pass/Fail).
- Execute Test Cases Manually: Open the web application in a browser and manually follow each test step.
- Log Results & Defects:
- Mark status as PASS if Expected Result == Actual Result.
- Mark status as FAIL if there is a mismatch and log a corresponding defect in the Defect Report sheet.

Sheet 1: Manual Test Cases & Execution Report

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_LOGIN_01	Verify valid login	1. Navigate to login page 2. Enter valid usern ame	**Username:** student **Password:** Password123	User successfully logged in; redirected to welcome page displaying "Logged In Successfully".	User logged in success fully and redirect ed to welcom e page.	**PASS**

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
		3. Enter valid password 4. Click Submit				

TC_LOGIN_02	Verify invalid password	1. Navigate to login page 2. Enter valid username 3. Enter invalid password	**Username:** student **Password:** WrongPass	Error message displayed : "Your password is invalid!".	Error message displayed: "Your password is invalid!".	**PASS**

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
		4. Click Submit				

TC_LOGIN_03	Verify blank fields submission	1. Navigate to login page 2. Leave username & password empty 3. Click Submit	**Username:** *[empty]* **Password:** *[em*

Sheet 1: Manual Test Cases & Execution Report

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_LOGIN_01	Verify valid login	1. Navigate to login page 2. Enter valid username 3. Enter valid password 4. Click Submit	**Username:** student **Password:** Password123	User successfully logged in; redirected to welcome page displaying "Logged In Successfully".	User logged in successfully and redirected to welcome page.	PASS
TC_LOGIN_02	Verify invalid password	page 2. Enter valid username 3. Enter invalid password 4. Click Submit	student **Password:** WrongPass	Error message displayed: "Your password is invalid!".	Error message displayed: "Your password is invalid!".	PASS

Sheet 2: Defect Report

Defect ID	Associated Test Case	Defect Description	Severity	Priority	Status
DEF_01	TC_LOGIN_03	System allows clicking submit with empty username and password without displaying field validation messages.	High		

MetricsTotal

Test Cases Executed: 3Passed: 2Failed: 1Pass Percentage: 66.6%

Conclusion

The Test Plan, Test Scenarios, and Test Cases were successfully designed and manually executed using Microsoft Excel. The Test Execution Report and Defect Report were generated as per standard STLC guidelines.

Practical No. 4

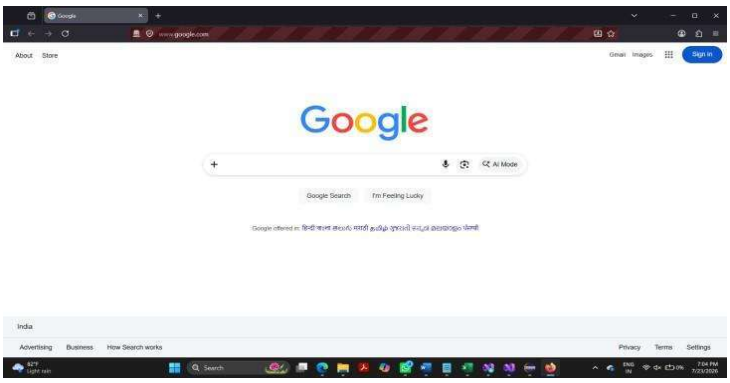
Install selenium Server(Selenium RC)and demonstrate its usage by excuting a script in java to automate browser actions.

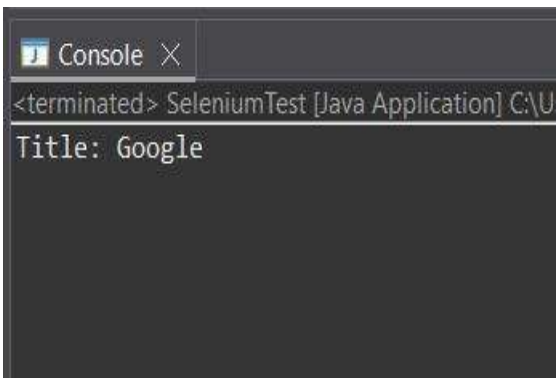
Code:

SeleniumTest.java | LoginTest.java | SeleniumGridDemo.java

```
1  package program;
2
3  import org.openqa.selenium.WebDriver;
5
6  public class SeleniumTest {
7
8      public static void main(String[] args) {
9
10         WebDriver driver = new FirefoxDriver();
11
12         driver.manage().window().maximize();
13
14         driver.get("https://www.google.com");
15
16         System.out.println("Title: " + driver.getTitle());
17
18         driver.quit();
19     }
20 }
21
```

Output:





Practical No. 5: Selenium Server (Grid) Configuration and Remote Execution

Aim: To configure Selenium Server (Grid) using a Hub-Node/Standalone architecture and execute automated test scripts on a remote browser using RemoteWebDriver.

SeleniumTest.java | LoginTest.java | *SeleniumGridDemo.java

```
package program;
import java.net.MalformedURLException;

public class SeleniumGridDemo {

    public static void main(String[] args) {
        try {

            ChromeOptions options = new ChromeOptions();


            URL hubUrl = new URL("http://localhost:4444");


            WebDriver driver = new RemoteWebDriver(hubUrl, options);


            System.out.println("Starting execution on Remote Grid...");
            driver.get("https://www.google.com");

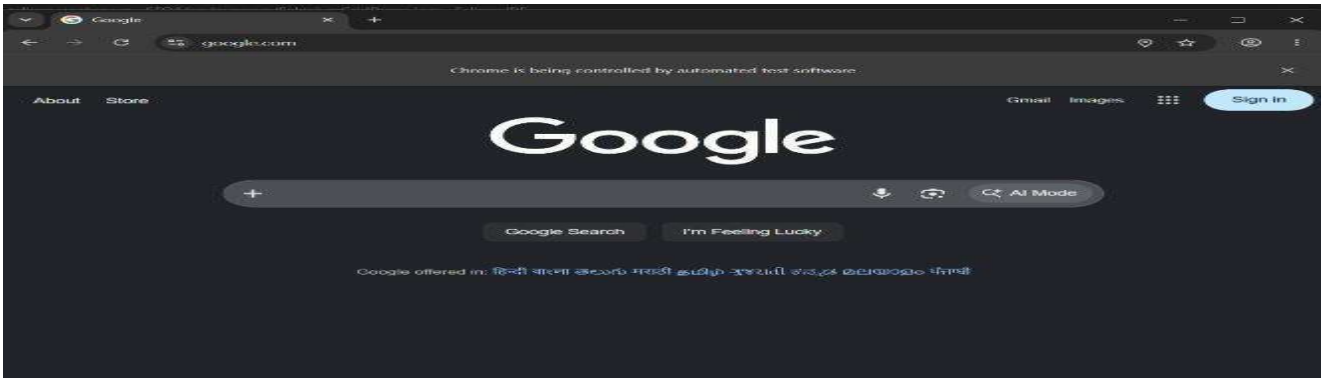

            String pageTitle = driver.getTitle();
            System.out.println("Page Title retrieved from Remote Browser: " + pageTitle);

            if (pageTitle.equals("Google")) {
                System.out.println("Test Status: PASSED");
            } else {
                System.out.println("Test Status: FAILED");
            }

            // Step 5: Close remote browser session
            //driver.quit();

        } catch (MalformedURLException e) {
            System.err.println("Invalid Hub URL format: " + e.getMessage());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:




```
Console
<terminated> SeleniumGridDemo [Java Application] C:\Users\Admin\.p2\p
Jul 23, 2026 7:08:24 PM org.openqa.selenium.remote.tra
INFO: Using OpenTelemetry for tracing
Starting execution on Remote Grid...
Page Title retrieved from Remote Browser: Google
Test Status: PASSED
```

Practical No.6: Login Automation and Validation

Aim: To write and execute an automated test script using Selenium WebDriver with Java to perform user login on a specified web application, and validate successful authentication using assertions like URL validation, welcome message verification, and logout button visibility checks.

SeleniumTest.java | LoginTest.java | *SeleniumGridDemo.java

```java
package program;

import org.openqa.selenium.By;

public class LoginTest {

    public static void main(String[] args) {

        WebDriver driver = new FirefoxDriver();

        driver.get("https://practicetestautomation.com/practice-test-login/");

        driver.findElement(By.id("username"))
                .sendKeys("student");

        driver.findElement(By.id("password"))
                .sendKeys("Password123");

        driver.findElement(By.id("submit"))
                .click();

        String url = driver.getCurrentUrl();

        if(url.contains("logged-in-successfully")) {

            System.out.println("URL Validation Passed");

        } else {

            System.out.println("URL Validation Failed");
        }

        String message = driver.findElement(By.tagName("h1")).getText();

        if(message.equals("Logged In Successfully")) {

            System.out.println("Welcome Message Passed");

        }

        if(driver.findElement(By.linkText("Log out")).isDisplayed()) {

            System.out.println("Logout Button Present");

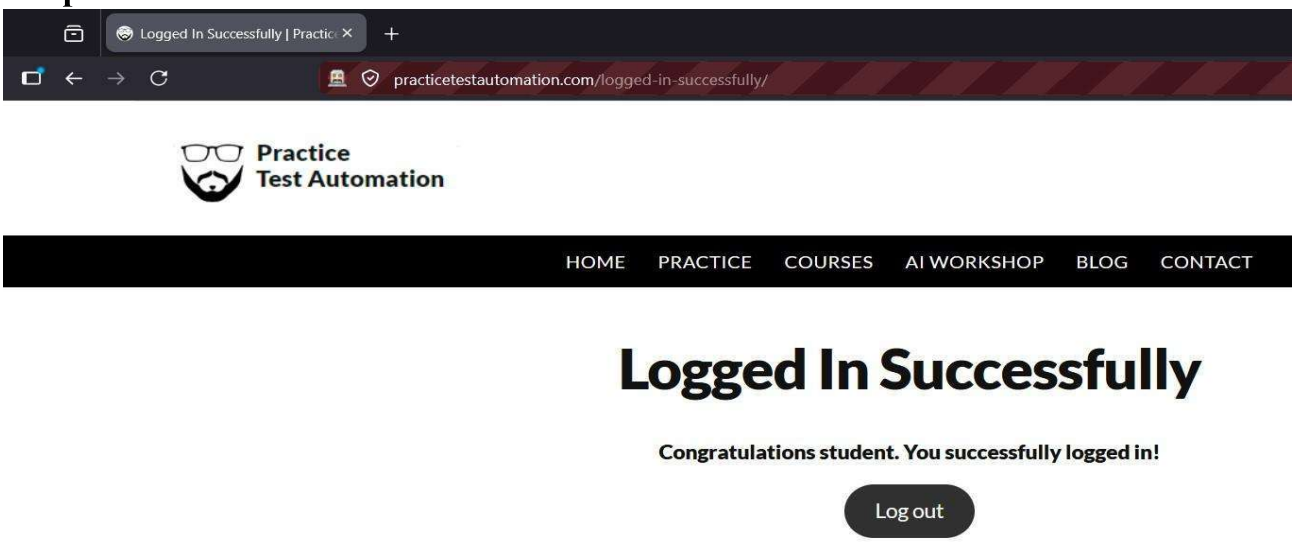
        }

        //driver.quit();

    }

}
```

Output:

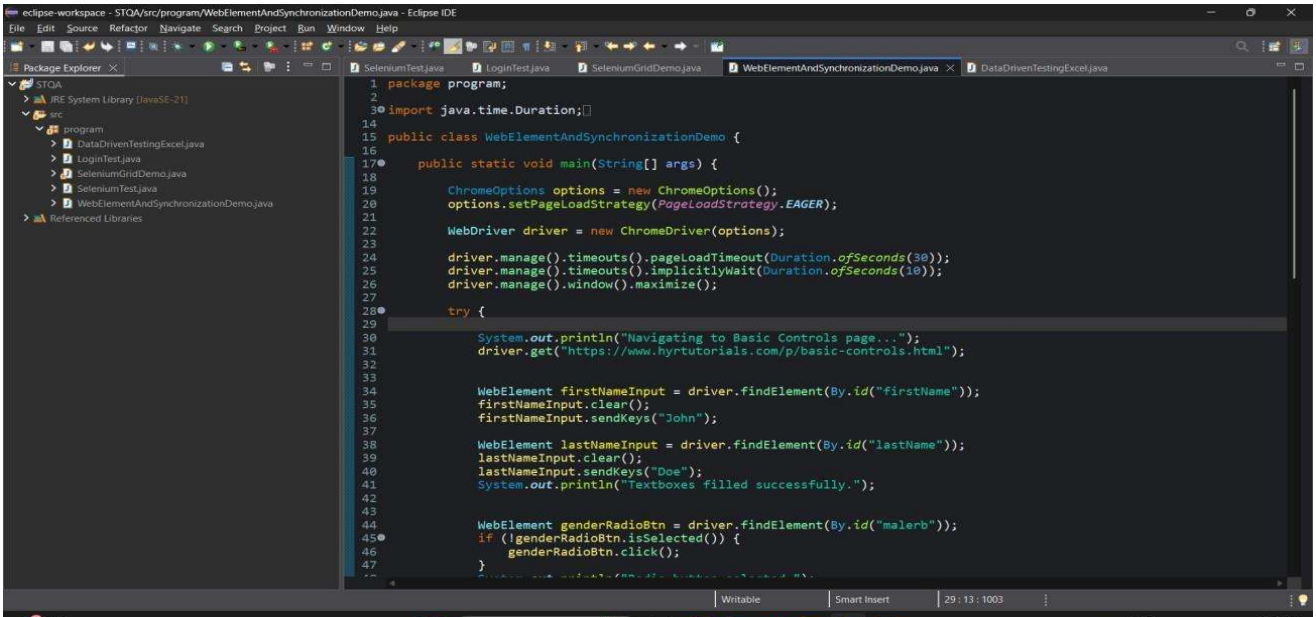

Logged In Successfully | Practic
practicetestautomation.com/logged-in-successfully/
Practice
Test Automation
HOME PRACTICE COURSES AI WORKSHOP BLOG CONTACT
Logged In Successfully
Congratulations student. You successfully logged in!
Log out

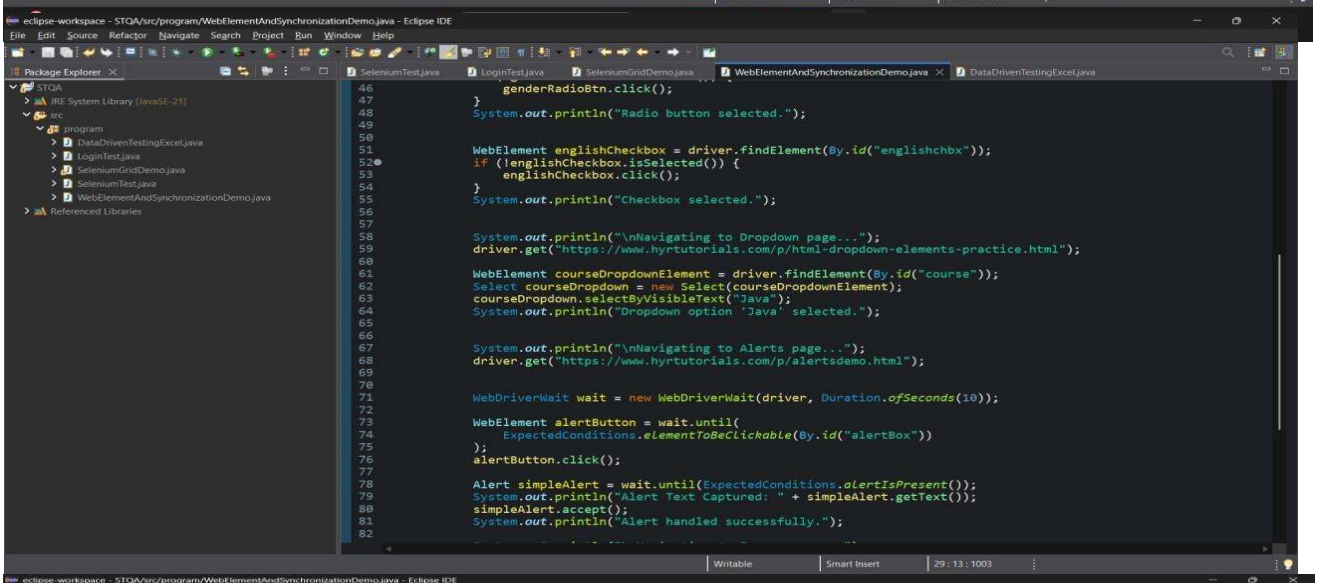

Console
<terminated> LoginTest [Java Application] C:\Users\Ac
URL Validation Passed
Welcome Message Passed
Logout Button Present

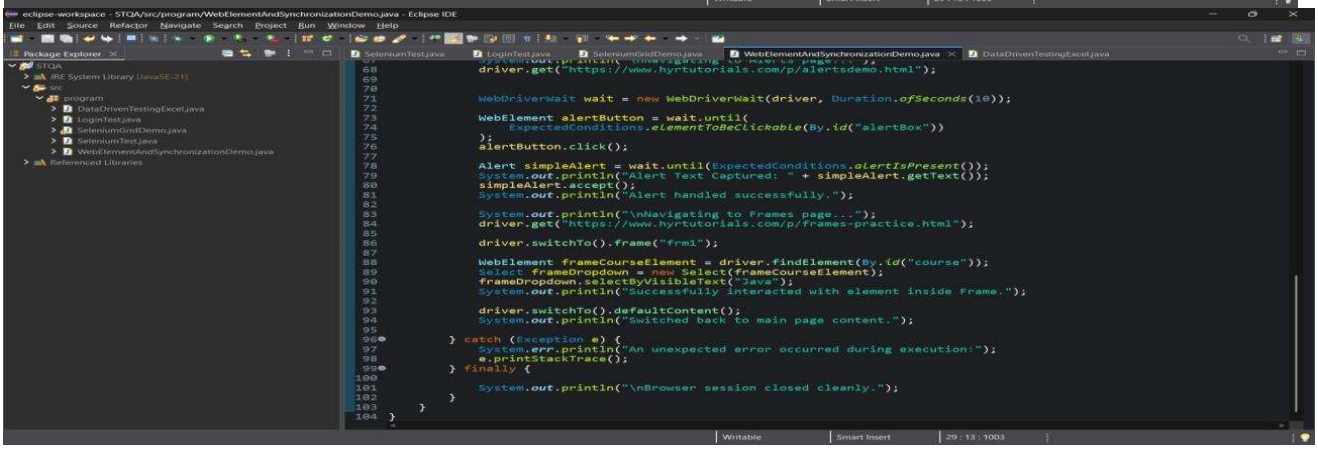
Practical No. 7

Aim: Web Element Interaction and Synchronization Handling

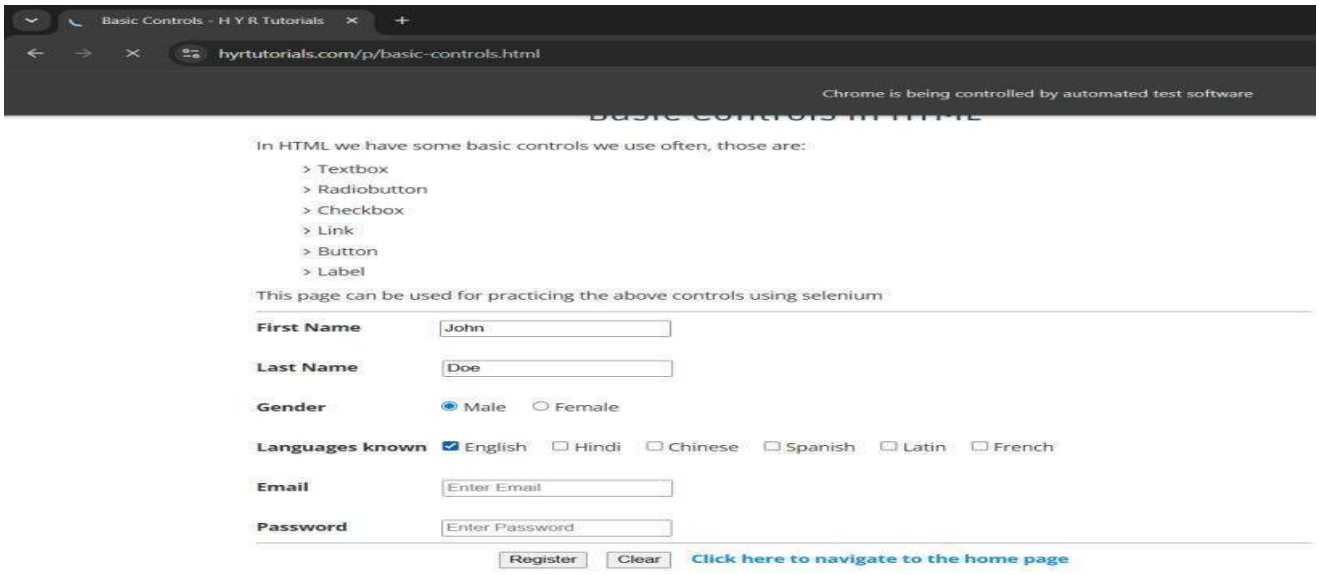
To develop a Java automation script using Selenium WebDriver to interact with various web elements (textboxes, radio buttons, checkboxes, dropdowns, alerts, and frames) and implement implicit and explicit waits for synchronization handling.

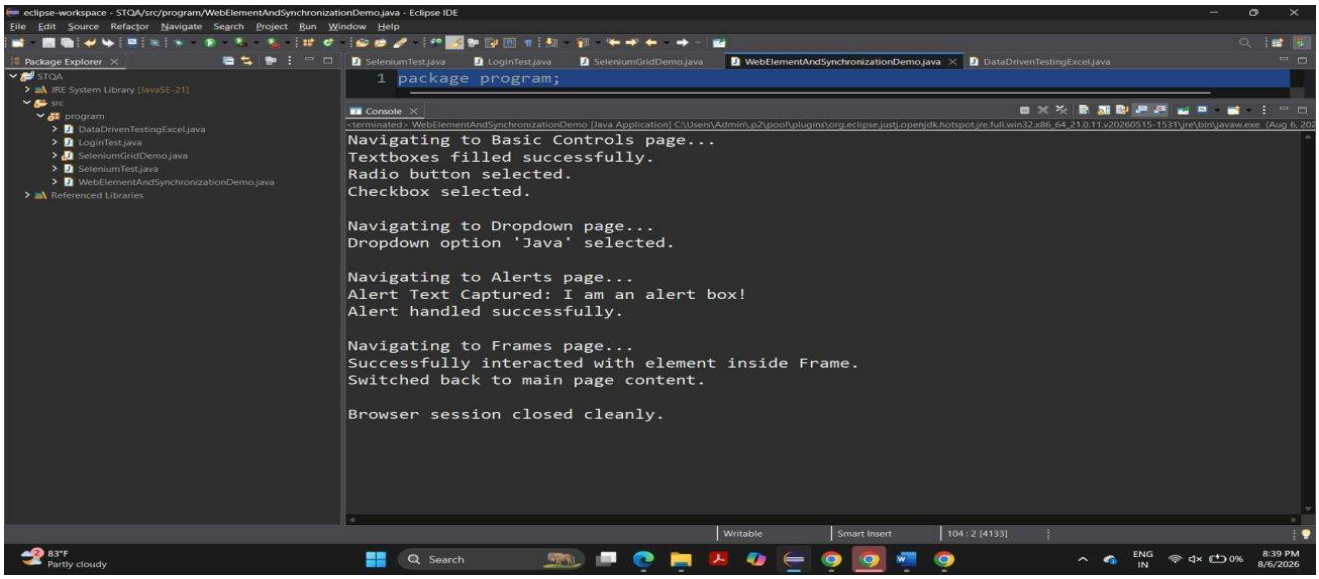






Output:


Basic Controls - H Y R Tutorials
hyrtutorials.com/p/basic-controls.html
Chrome is being controlled by automated test software
In HTML we have some basic controls we use often, those are:
> Textbox
> Radiobutton
> Checkbox
> Link
> Button
> Label
This page can be used for practicing the above controls using selenium
First Name
John
Last Name
Doe
Gender
Male
Female
Languages known
English
Hindi
Chinese
Spanish
Latin
French
Email
Enter Email
Password
Enter Password
Register
Clear
Click here to navigate to the home page


eclipse-workspace - STQA/src/program/WebElementAndSynchronizationDemo.java - Eclipse IDE
Navigating to Basic Controls page...
Textboxes filled successfully.
Radio button selected.
Checkbox selected.
Navigating to Dropdown page...
Dropdown option 'Java' selected.
Navigating to Alerts page...
Alert Text Captured: I am an alert box!
Alert handled successfully.
Navigating to Frames page...
Successfully interacted with element inside Frame.
Switched back to main page content.
Browser session closed cleanly.

Practical No.8 : Data-Driven Testing Using Excel Integration

Aim: To implement a Data-Driven Testing framework by developing a Java program using Selenium WebDriver and the Apache POI library to read 10 student records from an Excel file, enter and validate the records on a web application, update the test results back into the Excel file, and perform data verification.

Software & Library Requirements

- **JDK:** Java Development Kit (JDK 11 or higher)
- **IDE:** Eclipse IDE
- **Libraries Required (JARs):**
  - Selenium Java Client Webdriver JARs
  - **Apache POI** (poi, poi-ooxml, poi-ooxml-full JARs for .xlsx processing)
- **Web Browser:** Google Chrome / Firefox

Phase 1: Require JAR Files Setup

Make sure all **9 external JAR files** are present in your Eclipse project's **Build Path**:

1. poi-5.2.3.jar
2. poi-ooxml-5.2.3.jar
3. poi-ooxml-full-5.2.3.jar
4. commons-collections4-4.4.jar
5. xmlbeans-5.1.1.jar
6. commons-io-2.11.0.jar
7. commons-compress-1.21.jar
8. log4j-api-2.17.2.jar
9. log4j-core-2.17.2.jar *(Silences the log warning)*

Adding JARs In Eclipse:;

1. Right-click your project in **Package Explorer $\rightarrow$ Build Path $\rightarrow$ Configure Build Path...**
2. Select the **Libraries** tab (click **Classpath** if visible).
3. Click **Add External JARs...** on the right.
4. Select all 9 JAR files from your folder and click **Open**.
5. Click **Apply and Close**.

Phase 2: Excel File Setup

1. Create a folder named **Selenium1** on your **C:** drive (**C:\Selenium1**).
2. Create an Excel file inside that folder named StudentData.xlsx.
3. Set up your columns in Row 1: ○ **Cell A1: Student ID** ○ **Cell B1: Student Name** ○ **Cell C1: Marks** ○ **Cell D1: Test Result**

4. Fill in rows 2 through 11 with student data (leave **Column D completely blank**).
5. Press **Ctrl + S** to save the Excel file.
6. **Close Microsoft Excel completely** so Java can access the file.

CODE:

```
package program;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.time.Duration;
import org.apache.poi.ss.usermodel.Cell;
import org.apache.poi.ss.usermodel.CellType;
import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.xssf.usermodel.XSSFSheet;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.openqa.selenium.By;
import org.openqa.selenium.PageLoadStrategy;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class DataDrivenTestingExcel {

    public static void main(String[] args) {

        String excelFilePath = "C:/Selenium1/StudentData.xlsx";

        WebDriver driver = null;
        FileInputStream fis = null;
        FileOutputStream fos = null;
        XSSFWorkbook workbook = null;

        try {

            // Step 1: Initialize Excel Input Stream using Apache POI
            fis = new FileInputStream(excelFilePath);
            workbook = new XSSFWorkbook(fis);

            // Get the first sheet directly by index
            XSSFSheet sheet = workbook.getSheetAt(0);

            // Calculate total rows safely
            int totalRows = sheet.getLastRowNum();

            if (totalRows <= 0) {
                totalRows = sheet.getPhysicalNumberOfRows() - 1;
            }

            System.out.println("Total Student Records Found: " + totalRows);

            if (totalRows <= 0) {
                System.err.println("No records found in Excel. Please verify
C:/Selenium1/StudentData.xlsx.");
                return;
            }

            // Step 2: Configure and Launch Browser
            ChromeOptions options = new ChromeOptions();
            options.setPageLoadStrategy(PageLoadStrategy.EAGER);
            driver = new ChromeDriver(options);

            driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
            driver.manage().window().maximize();
```

```
            // Step 3: Iterate through Student Records (Row 1 to Row 10)
            for (int i = 1; i <= totalRows; i++) {

                Row row = sheet.getRow(i);

                if (row == null) continue;

                // Extract Cell values safely
                Cell idCell = row.getCell(0);
                Cell nameCell = row.getCell(1);
                Cell marksCell = row.getCell(2);

                if (idCell == null || nameCell == null || marksCell == null) continue;

                String studentId = (idCell.getCellType() == CellType.STRING)
                        ? idCell.getStringCellValue()
                        : String.valueOf((int) idCell.getNumericCellValue());

                String studentName = nameCell.getStringCellValue();

                double marks = (marksCell.getCellType() == CellType.NUMERIC)
                        ? marksCell.getNumericCellValue()
                        : Double.parseDouble(marksCell.getStringCellValue());

                System.out.println("\n--- Processing Student " + i + ": " + studentId +
" ---");

                // Navigate to target form
                driver.get("https://the-internet.herokuapp.com/inputs");

                // Enter data into input field
                WebElement inputField = driver.findElement(By.tagName("input"));
                inputField.clear();
                inputField.sendKeys(String.valueOf((int) marks));

                // Verification logic
                String enteredValue = inputField.getAttribute("value");
                Cell resultCell = row.createCell(3);

                if (enteredValue.equals(String.valueOf((int) marks))) {

                    System.out.println("Validation Successful for " + studentName);
                    resultCell.setCellValue("PASSED");

                } else {

                    System.out.println("Validation Failed for " + studentName);
                    resultCell.setCellValue("FAILED");
                }
            }

            // Step 4: Close Input Stream before saving back
            fis.close();
            fis = null;

            // Step 5: Write Results back into Excel File
            fos = new FileOutputStream(excelFilePath);
            workbook.write(fos);

            System.out.println("\nExcel File Updated Successfully with Test Results!");

        } catch (Exception e) {

            System.err.println("Error occurred during execution:");
            e.printStackTrace();
```

```
        } finally {

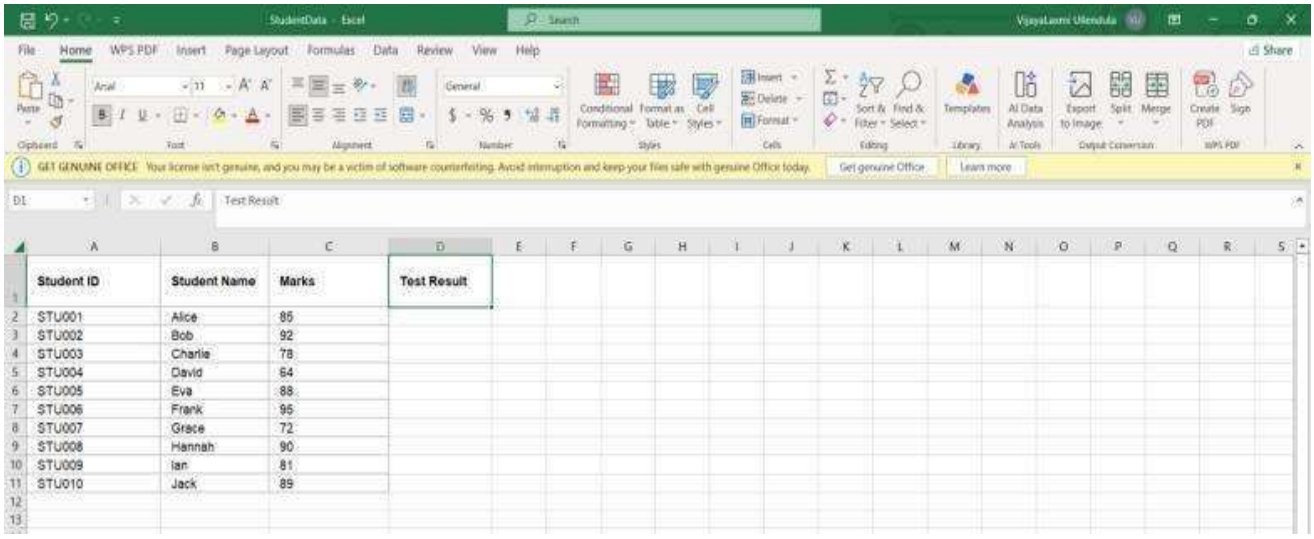
            try {

                if (fis != null) fis.close();
                if (fos != null) fos.close();
                if (workbook != null) workbook.close();
                if (driver != null) driver.quit();

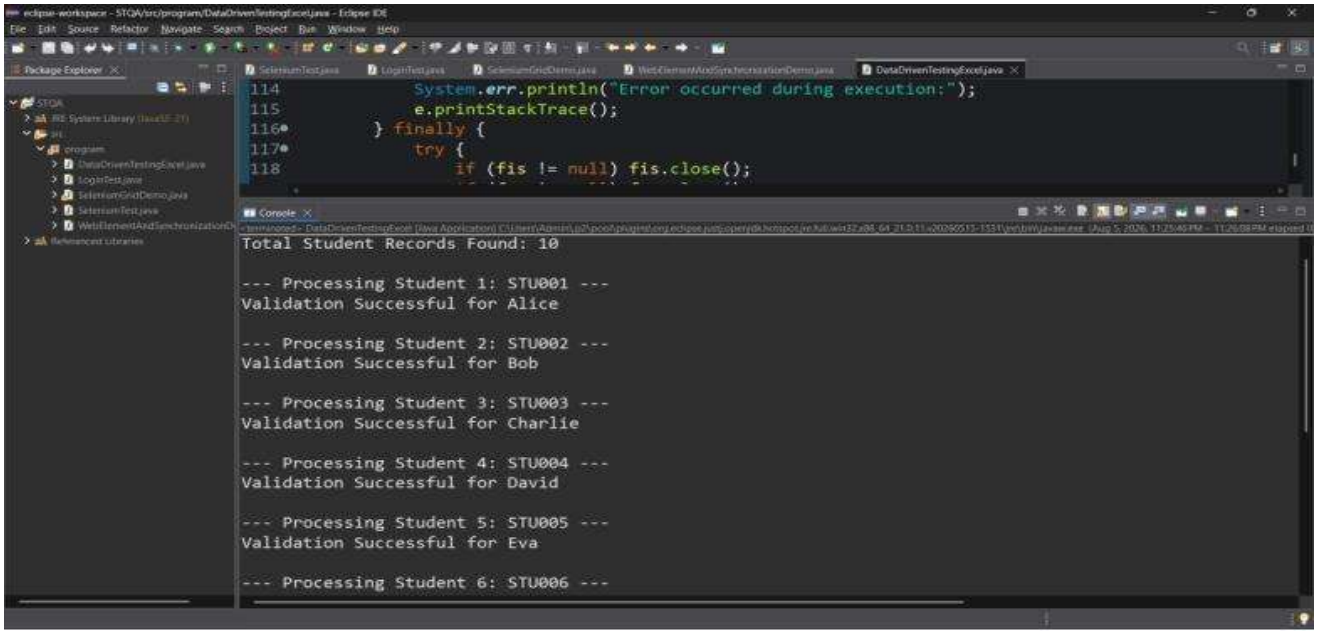
            } catch (Exception ex) {

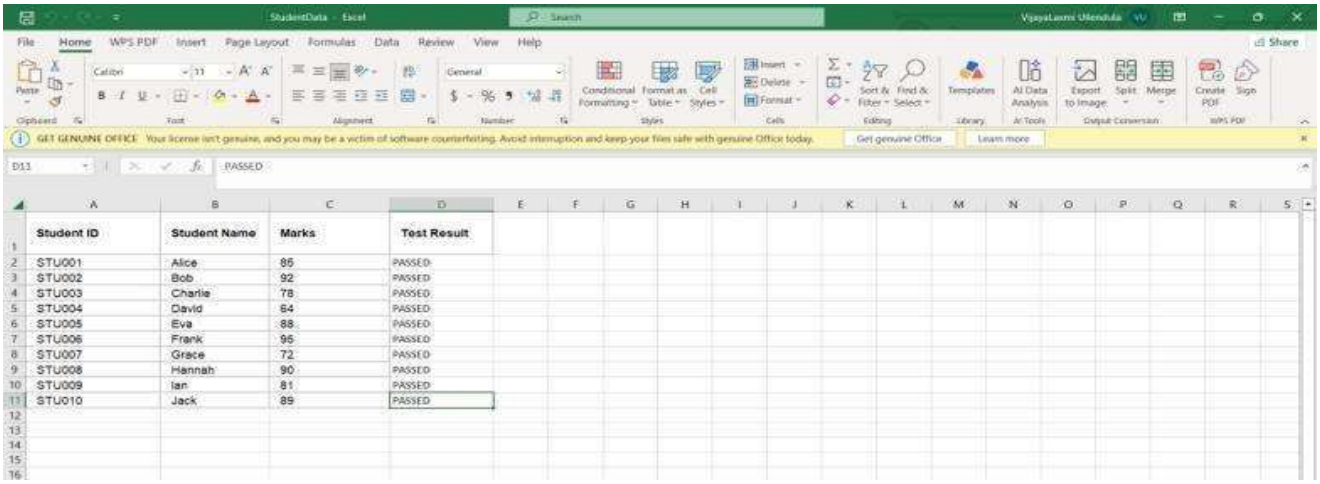
                ex.printStackTrace();
            }
        }
    }
}
```

OUTPUT:



Student ID	Student Name	Marks	Test Result
STU001	Alice	85	
STU002	Bob	92	
STU003	Charlie	78	
STU004	David	64	
STU005	Eva	88	
STU006	Frank	95	
STU007	Grace	72	
STU008	Hannah	90	
STU009	Ian	81	
STU010	Jack	89	





	A	B	C	D
1	**Student ID**	**Student Name**	**Marks**	**Test Result**
2	STU001	Alice	85	PASSED
3	STU002	Bob	92	PASSED
4	STU003	Charlie	78	PASSED
5	STU004	David	64	PASSED
6	STU005	Eva	88	PASSED
7	STU006	Frank	95	PASSED
8	STU007	Grace	72	PASSED
9	STU008	Hannah	90	PASSED
10	STU009	Ian	81	PASSED
11	STU010	Jack	89	PASSED

Practical No. 9: Data Extraction and Score Analysis from Web Table

Aim: Automate the extraction of student marks from a dynamic web table, identify students scoring above 60 in at least one subject, and compute the total count and percentage.

CODE:

```
package program;

import java.time.Duration;
import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class WebTable {

    public static void main(String[] args) {


java.util.logging.Logger.getLogger("org.openqa.selenium").setLevel(java.util.logging.Le
vel.OFF);

        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();

        try {

            // Target web page containing a student score table
            driver.get("https://the-internet.herokuapp.com/tables");

            // Locate table rows (excluding table header)
            List<WebElement> rows =
                    driver.findElements(By.xpath("//table[@id='table1']/tbody/tr"));

            int totalStudents = rows.size();
            int qualifyingStudents = 0;

            System.out.println("=== Practical 9: Web Table Analysis ===");
            System.out.println("Total Students Evaluated: " + totalStudents);

            for (int i = 0; i < totalStudents; i++) {

                List<WebElement> cells = rows.get(i).findElements(By.tagName("td"));

                // Extract student name and score column (e.g., Due Amount / Score
cell)
                String studentName = cells.get(1).getText() + " " +
cells.get(0).getText();
                String rawScore = cells.get(3).getText().replace("$", "").trim();
                double score = Double.parseDouble(rawScore);

                // Condition: Score above 60
                if (score > 60.0) {
                    qualifyingStudents++;

                    System.out.println("Student Passed Condition (>60): " + studentName
+ " | Score: " + score);
                }
```

```
            }

            // Calculate percentage
            double percentage = ((double) qualifyingStudents / totalStudents) * 100;

            System.out.println("\n--- Summary Report ---");
            System.out.println("Students Scoring Above 60: " + qualifyingStudents);
            System.out.println("Percentage of Qualifying Students: " +
                    String.format("%.2f", percentage) + "%");

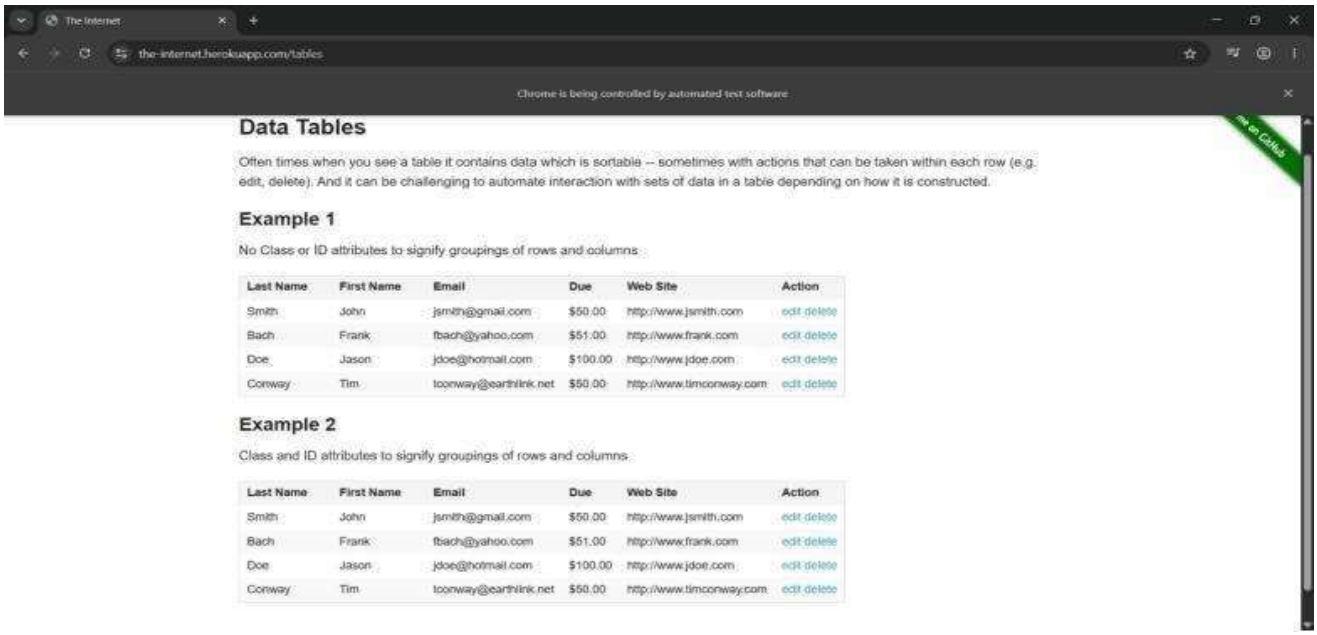
        } catch (Exception e) {

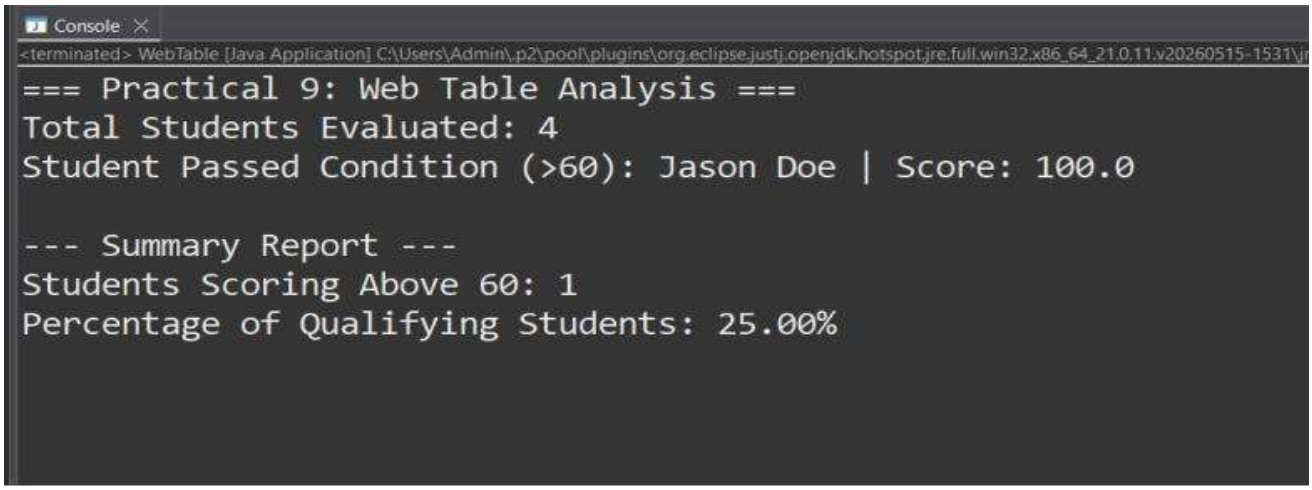
            e.printStackTrace();

        } finally {

            //driver.quit();
        }
    }
}
```

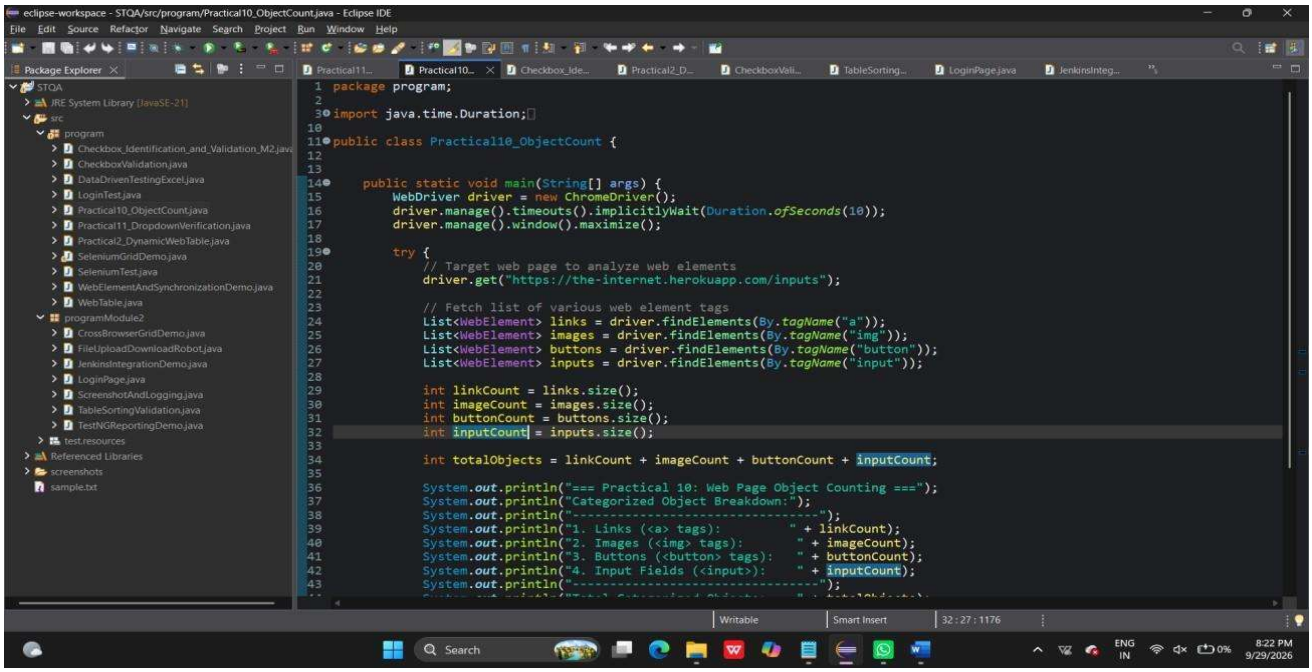
OUTPUT;





Practical No. 10: List and Combo Box Item Verification

Aim: To automate the process of identifying a list or dropdown box on a web page, determine total number of items, validate default selected value, and perform selection testing.



```
package program;

import java.time.Duration;

public class Practical10_ObjectCount {

    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();

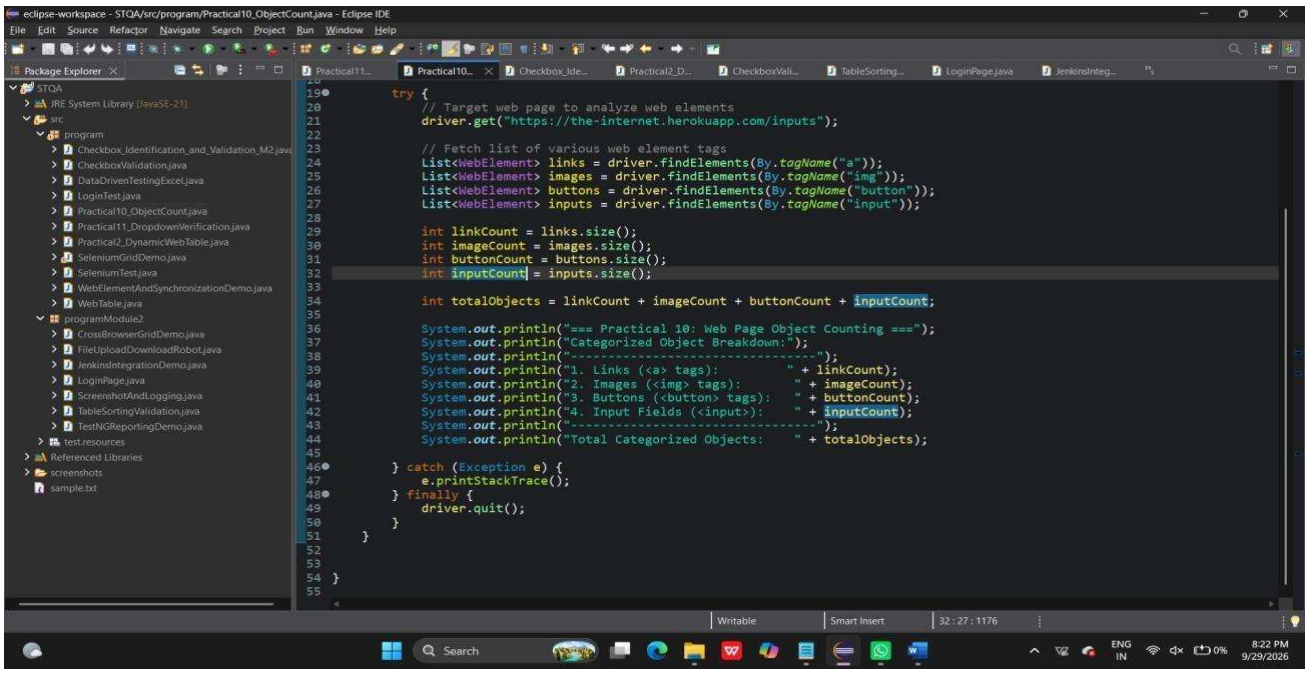
        try {
            // Target web page to analyze web elements
            driver.get("https://the-internet.herokuapp.com/inputs");

            // Fetch list of various web element tags
            List<WebElement> links = driver.findElements(By.tagName("a"));
            List<WebElement> images = driver.findElements(By.tagName("img"));
            List<WebElement> buttons = driver.findElements(By.tagName("button"));
            List<WebElement> inputs = driver.findElements(By.tagName("input"));

            int linkCount = links.size();
            int imageCount = images.size();
            int buttonCount = buttons.size();
            int inputCount = inputs.size();

            int totalObjects = linkCount + imageCount + buttonCount + inputCount;

            System.out.println("=== Practical 10: Web Page Object Counting ===");
            System.out.println("Categorized Object Breakdown:");
            System.out.println("---------------------------------");
            System.out.println("1. Links (<a> tags):          " + linkCount);
            System.out.println("2. Images (<img> tags):       " + imageCount);
            System.out.println("3. Buttons (<button> tags):   " + buttonCount);
            System.out.println("4. Input Fields (<input>):    " + inputCount);
            System.out.println("---------------------------------");
```



```
        try {
            // Target web page to analyze web elements
            driver.get("https://the-internet.herokuapp.com/inputs");

            // Fetch list of various web element tags
            List<WebElement> links = driver.findElements(By.tagName("a"));
            List<WebElement> images = driver.findElements(By.tagName("img"));
            List<WebElement> buttons = driver.findElements(By.tagName("button"));
            List<WebElement> inputs = driver.findElements(By.tagName("input"));

            int linkCount = links.size();
            int imageCount = images.size();
            int buttonCount = buttons.size();
            int inputCount = inputs.size();

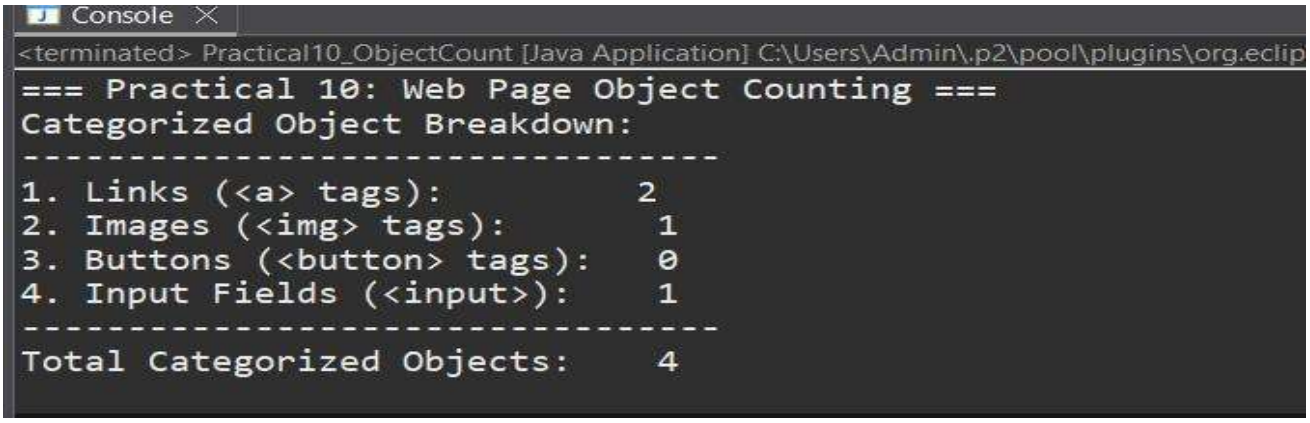
            int totalObjects = linkCount + imageCount + buttonCount + inputCount;

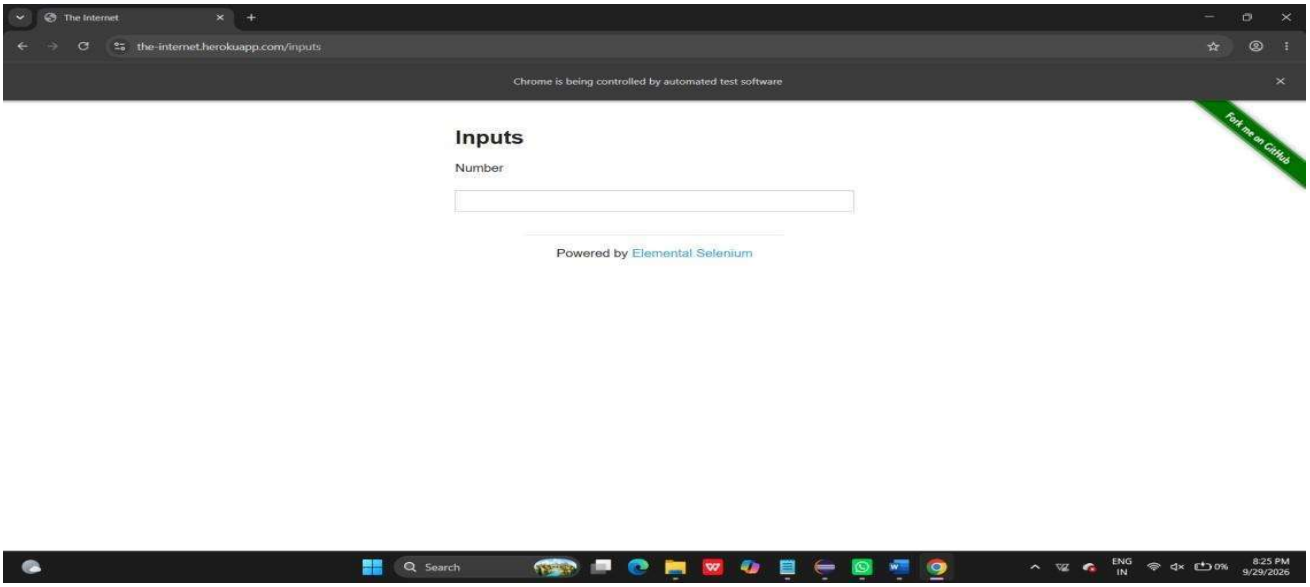
            System.out.println("=== Practical 10: Web Page Object Counting ===");
            System.out.println("Categorized Object Breakdown:");
            System.out.println("---------------------------------");
            System.out.println("1. Links (<a> tags):          " + linkCount);
            System.out.println("2. Images (<img> tags):       " + imageCount);
            System.out.println("3. Buttons (<button> tags):   " + buttonCount);
            System.out.println("4. Input Fields (<input>):    " + inputCount);
            System.out.println("---------------------------------");
            System.out.println("Total Categorized Objects:    " + totalObjects);

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            driver.quit();
        }
    }

}
```

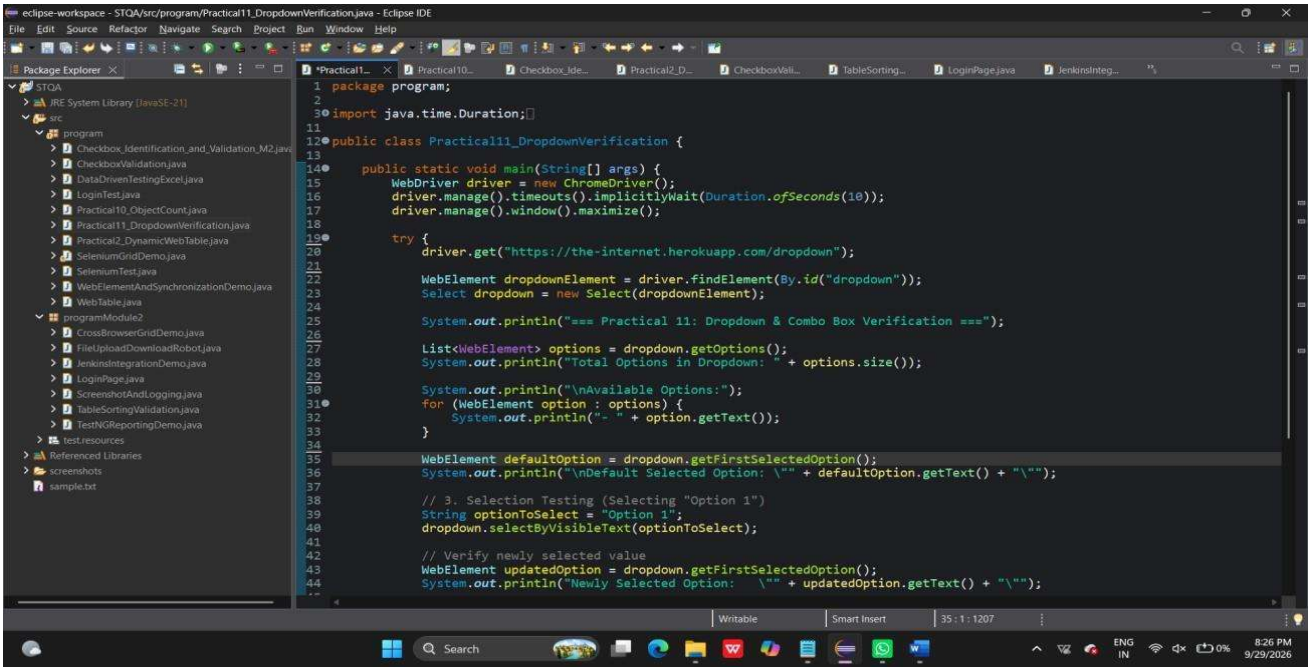
OUTPUT:


Console
<terminated> Practical10_ObjectCount [Java Application] C:\Users\Admin\.p2\pool\plugins\org.eclip
=== Practical 10: Web Page Object Counting ===
Categorized Object Breakdown:
1. Links (<a> tags): 2
2. Images (<img> tags): 1
3. Buttons (<button> tags): 0
4. Input Fields (<input>): 1
Total Categorized Objects: 4


The Internet
the-internet.herokuapp.com/inputs
Chrome is being controlled by automated test software
Inputs
Number
Powered by Elemental Selenium
Fork me on GitHub
Search

Practical No. 11: Checkbox Identification and Validation

Aim: To write a program to count total, checked, and unchecked checkboxes on a web page, and perform dynamic selection and validation.



```
package program;

import java.time.Duration;

public class Practical11_DropdownVerification {

    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();

        try {
            driver.get("https://the-internet.herokuapp.com/dropdown");

            WebElement dropdownElement = driver.findElement(By.id("dropdown"));
            Select dropdown = new Select(dropdownElement);

            System.out.println("=== Practical 11: Dropdown & Combo Box Verification ===");

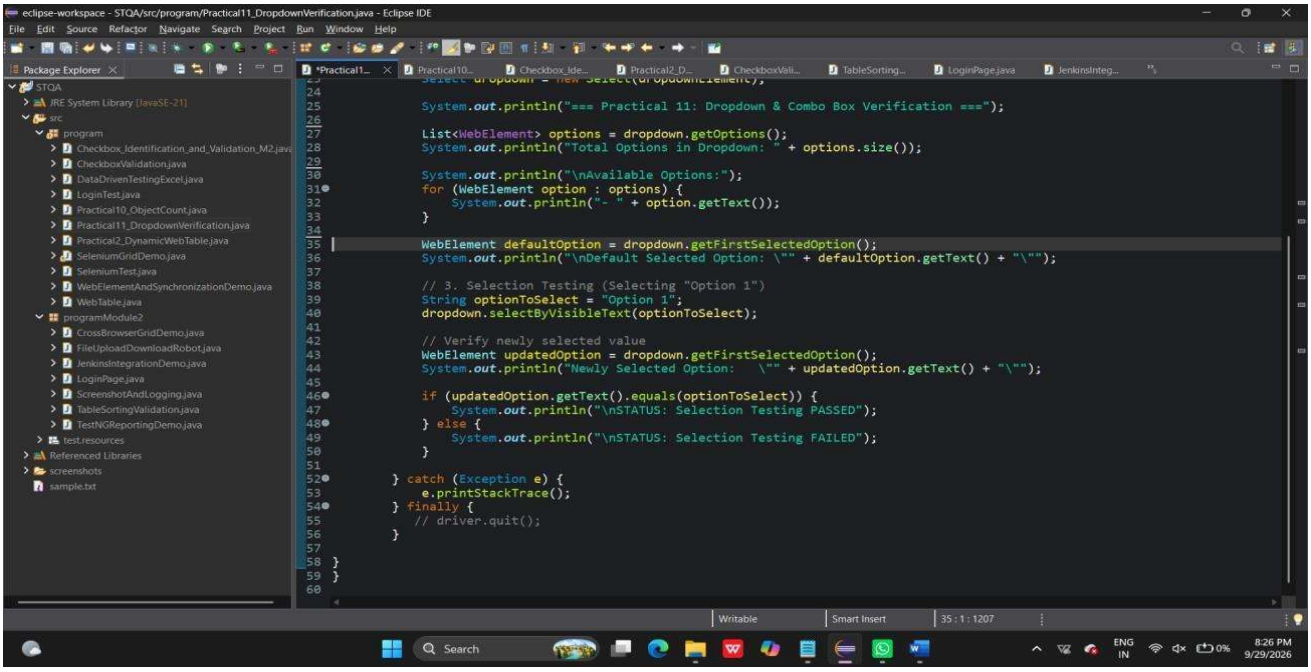
            List<WebElement> options = dropdown.getOptions();
            System.out.println("Total Options in Dropdown: " + options.size());

            System.out.println("\nAvailable Options:");
            for (WebElement option : options) {
                System.out.println("- " + option.getText());
            }

            WebElement defaultOption = dropdown.getFirstSelectedOption();
            System.out.println("\nDefault Selected Option: \"" + defaultOption.getText() + "\"");

            // 3. Selection Testing (Selecting "Option 1")
            String optionToSelect = "Option 1";
            dropdown.selectByVisibleText(optionToSelect);

            // Verify newly selected value
            WebElement updatedOption = dropdown.getFirstSelectedOption();
            System.out.println("Newly Selected Option:   \"" + updatedOption.getText() + "\"");
```



```
            System.out.println("=== Practical 11: Dropdown & Combo Box Verification ===");

            List<WebElement> options = dropdown.getOptions();
            System.out.println("Total Options in Dropdown: " + options.size());

            System.out.println("\nAvailable Options:");
            for (WebElement option : options) {
                System.out.println("- " + option.getText());
            }

            WebElement defaultOption = dropdown.getFirstSelectedOption();
            System.out.println("\nDefault Selected Option: \"" + defaultOption.getText() + "\"");

            // 3. Selection Testing (Selecting "Option 1")
            String optionToSelect = "Option 1";
            dropdown.selectByVisibleText(optionToSelect);

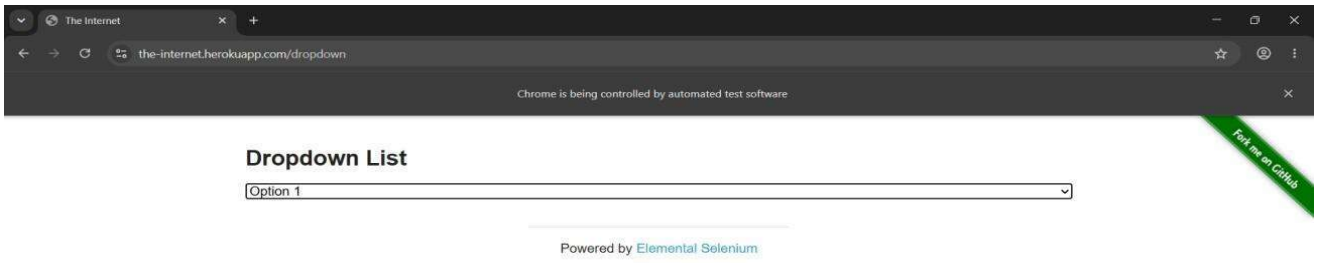
            // Verify newly selected value
            WebElement updatedOption = dropdown.getFirstSelectedOption();
            System.out.println("Newly Selected Option:   \"" + updatedOption.getText() + "\"");

            if (updatedOption.getText().equals(optionToSelect)) {
                System.out.println("\nSTATUS: Selection Testing PASSED");
            } else {
                System.out.println("\nSTATUS: Selection Testing FAILED");
            }

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            // driver.quit();
        }

}
}
```

OUTPUT:




```
=== Practical 11: Dropdown & Combo Box Verification ===
Total Options in Dropdown: 3

Available Options:
- Please select an option
- Option 1
- Option 2

Default Selected Option: "Please select an option"
Newly Selected Option:   "Option 1"

STATUS: Selection Testing PASSED
```

Practical No. 12: Dynamic Web Table Handling and Sorting Validation

```java
package programModule2;


import java.time.Duration;

public class TableSortingValidation {
    public static void main(String[] args) {

        java.util.logging.Logger.getLogger("org.openqa.selenium").setLevel(java.util.logging.Level.OFF);

        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();

        try {
            driver.get("https://the-internet.herokuapp.com/tables");

            System.out.println("=== Dynamic Web Table Sorting Validation ===");

            WebElement lastNameHeader = driver.findElement(By.xpath("//table[@id='table1']/thead/tr/th[1]"));
            lastNameHeader.click();

            // Extract values after column sort
            List<WebElement> rows = driver.findElements(By.xpath("//table[@id='table1']/tbody/tr"));
            List<String> actualSortedList = new ArrayList<>();

            for (WebElement row : rows) {
                String lastName = row.findElements(By.tagName("td")).get(0).getText();
                actualSortedList.add(lastName);
            }

            System.out.println("Extracted Table Data: " + actualSortedList);

            List<String> expectedSortedList = new ArrayList<>(actualSortedList);
            Collections.sort(expectedSortedList);

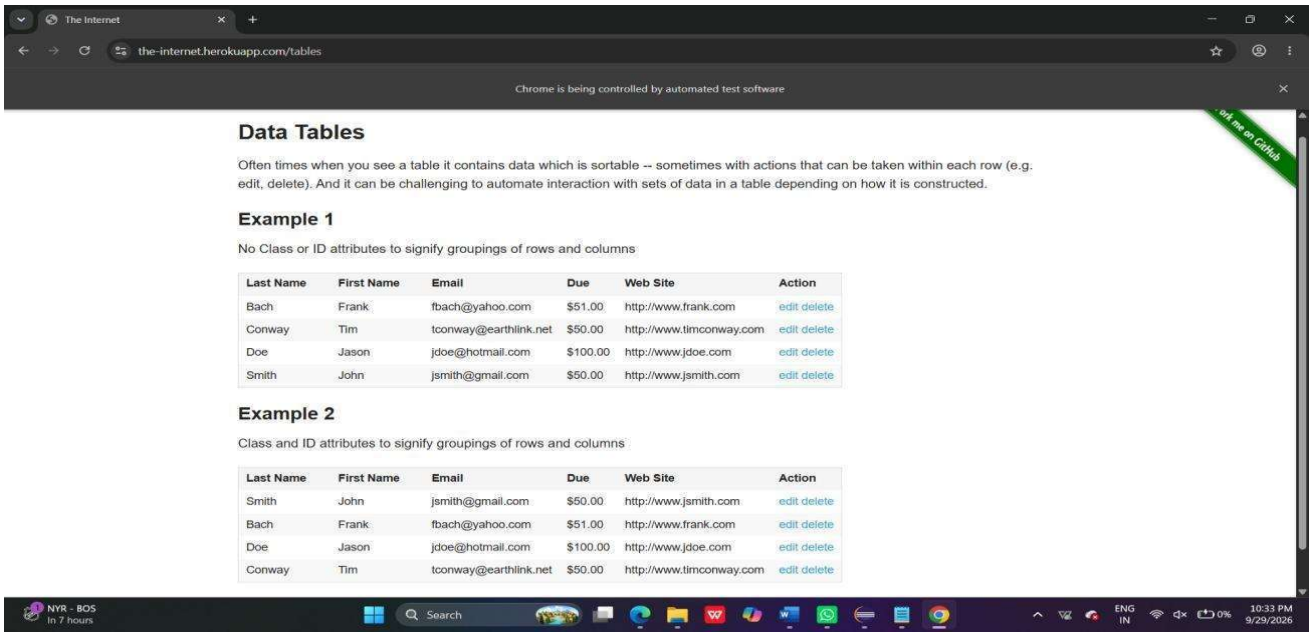
            System.out.println("Expected Sorted Data:  " + expectedSortedList);

            // Compare actual UI sorting against expected sorting
            boolean isSorted = actualSortedList.equals(expectedSortedList);

            System.out.println("\n--- Summary Report ---");
            System.out.println("Table Column Sorting Validation: " + (isSorted ? "PASS - Data is correctly sorted." : "FAIL - Sorting mismatch detected."));

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            //driver.quit();
        }
    }
}
```

OUTPUT:

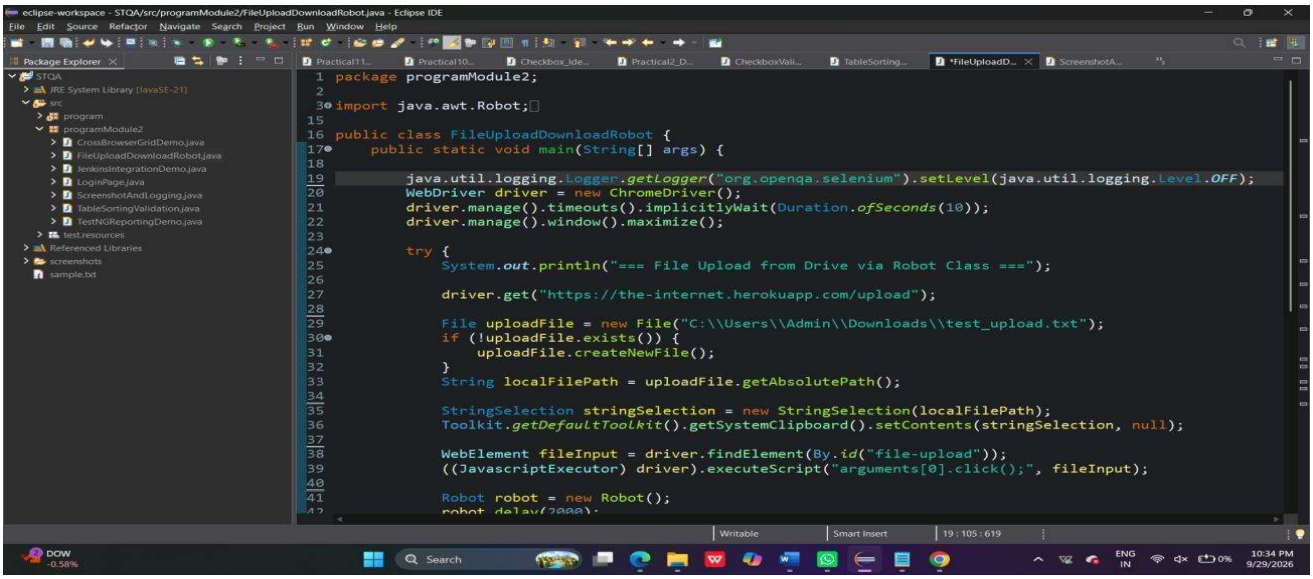



```
=== Dynamic Web Table Sorting Validation ===
Extracted Table Data: [Bach, Conway, Doe, Smith]
Expected Sorted Data:  [Bach, Conway, Doe, Smith]

--- Summary Report ---
Table Column Sorting Validation: PASS - Data is correctly sorted.
```

Practical No. 13: File Upload and Download Automation

Aim: To develop a script to automate file upload and download functionality, validate successful upload, and verify file existence after download using Selenium WebDriver and Robot Class.



```
package programModule2;

import java.awt.Robot;

public class FileUploadDownloadRobot {
    public static void main(String[] args) {

        java.util.logging.Logger.getLogger("org.openqa.selenium").setLevel(java.util.logging.Level.OFF);
        WebDriver driver = new ChromeDriver();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();

        try {
            System.out.println("=== File Upload from Drive via Robot Class ===");

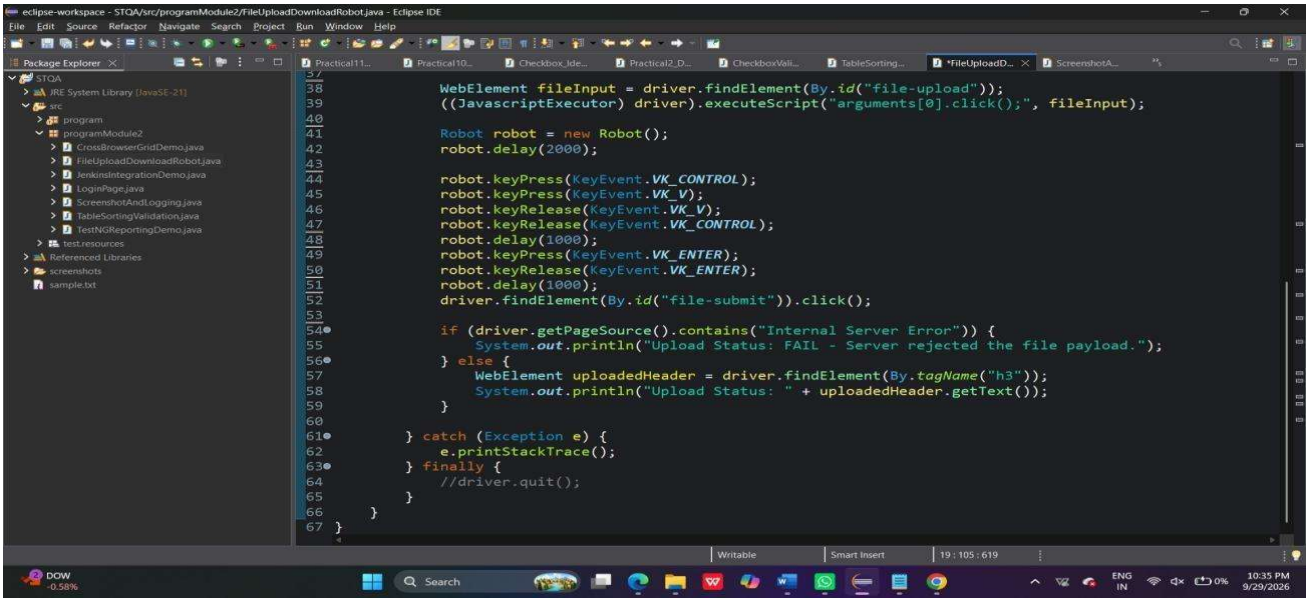
            driver.get("https://the-internet.herokuapp.com/upload");

            File uploadFile = new File("C:\\Users\\Admin\\Downloads\\test_upload.txt");
            if (!uploadFile.exists()) {
                uploadFile.createNewFile();
            }
            String localFilePath = uploadFile.getAbsolutePath();

            StringSelection stringSelection = new StringSelection(localFilePath);
            Toolkit.getDefaultToolkit().getSystemClipboard().setContents(stringSelection, null);

            WebElement fileInput = driver.findElement(By.id("file-upload"));
            ((JavascriptExecutor) driver).executeScript("arguments[0].click();", fileInput);

            Robot robot = new Robot();
```



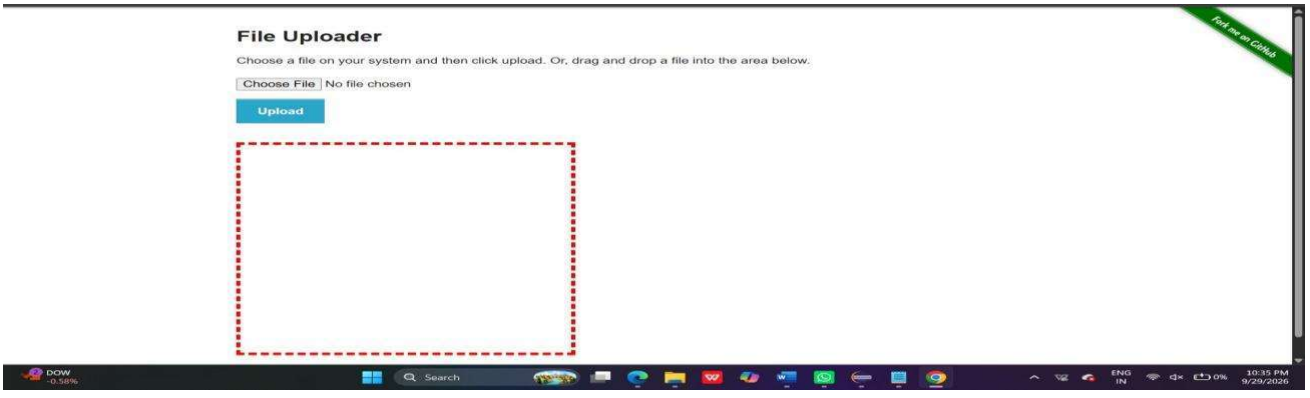
```
            WebElement fileInput = driver.findElement(By.id("file-upload"));
            ((JavascriptExecutor) driver).executeScript("arguments[0].click();", fileInput);

            Robot robot = new Robot();
            robot.delay(2000);

            robot.keyPress(KeyEvent.VK_CONTROL);
            robot.keyPress(KeyEvent.VK_V);
            robot.keyRelease(KeyEvent.VK_V);
            robot.keyRelease(KeyEvent.VK_CONTROL);
            robot.delay(1000);
            robot.keyPress(KeyEvent.VK_ENTER);
            robot.keyRelease(KeyEvent.VK_ENTER);
            robot.delay(1000);
            driver.findElement(By.id("file-submit")).click();

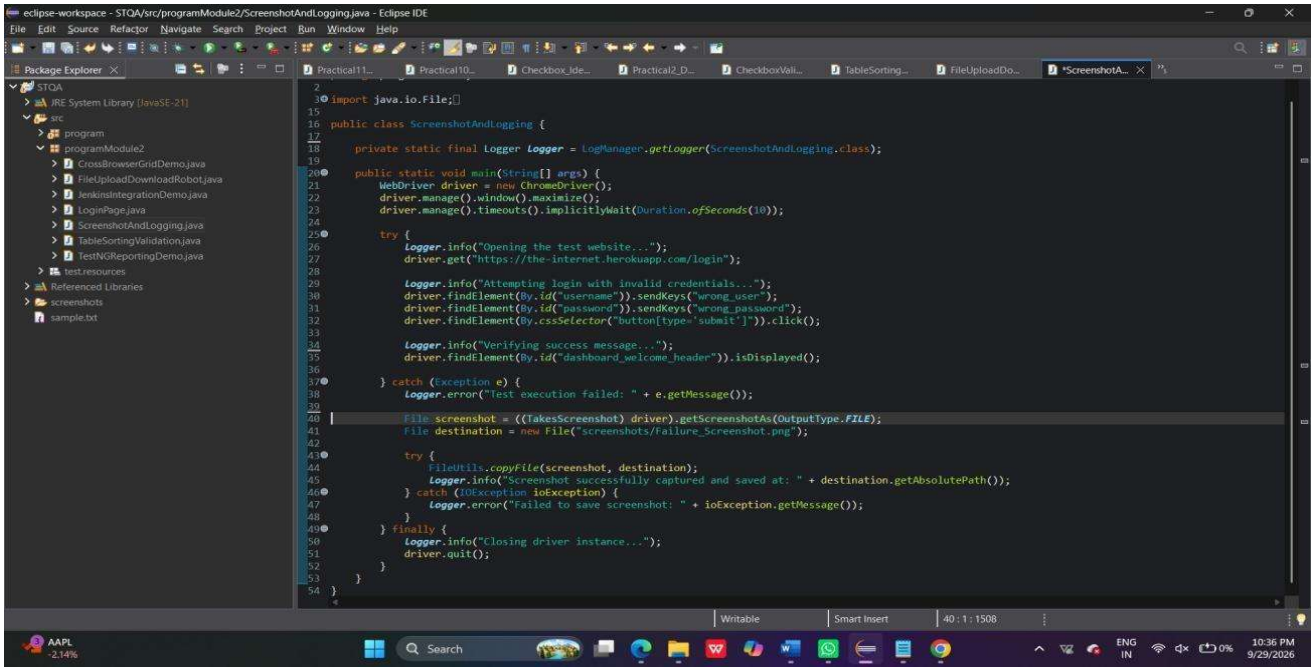
            if (driver.getPageSource().contains("Internal Server Error")) {
                System.out.println("Upload Status: FAIL - Server rejected the file payload.");
            } else {
                WebElement uploadedHeader = driver.findElement(By.tagName("h3"));
                System.out.println("Upload Status: " + uploadedHeader.getText());
            }

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            //driver.quit();
        }
    }
}
```

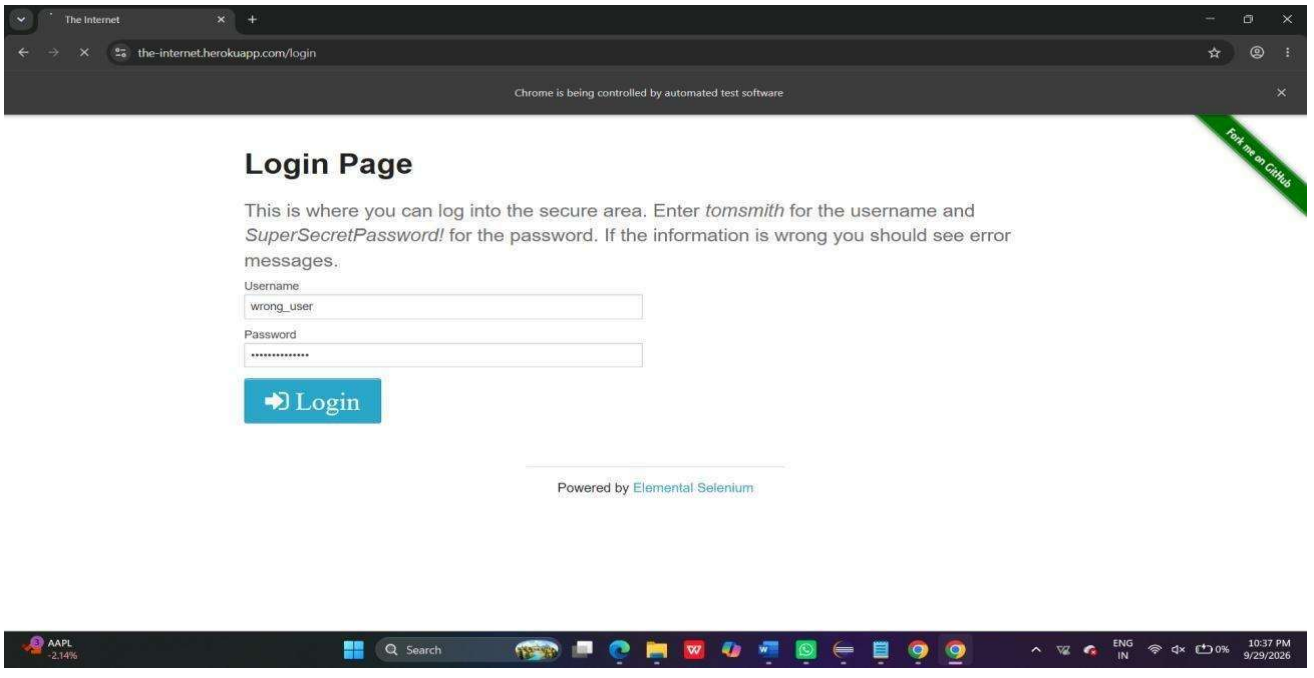


Practical No.14: Screenshot Capture and Logging Mechanism (Log4j)

Aim: To implement screenshot capture on test failure and integrate logging mechanism using Log4j, generating execution logs and analyzing error messages.

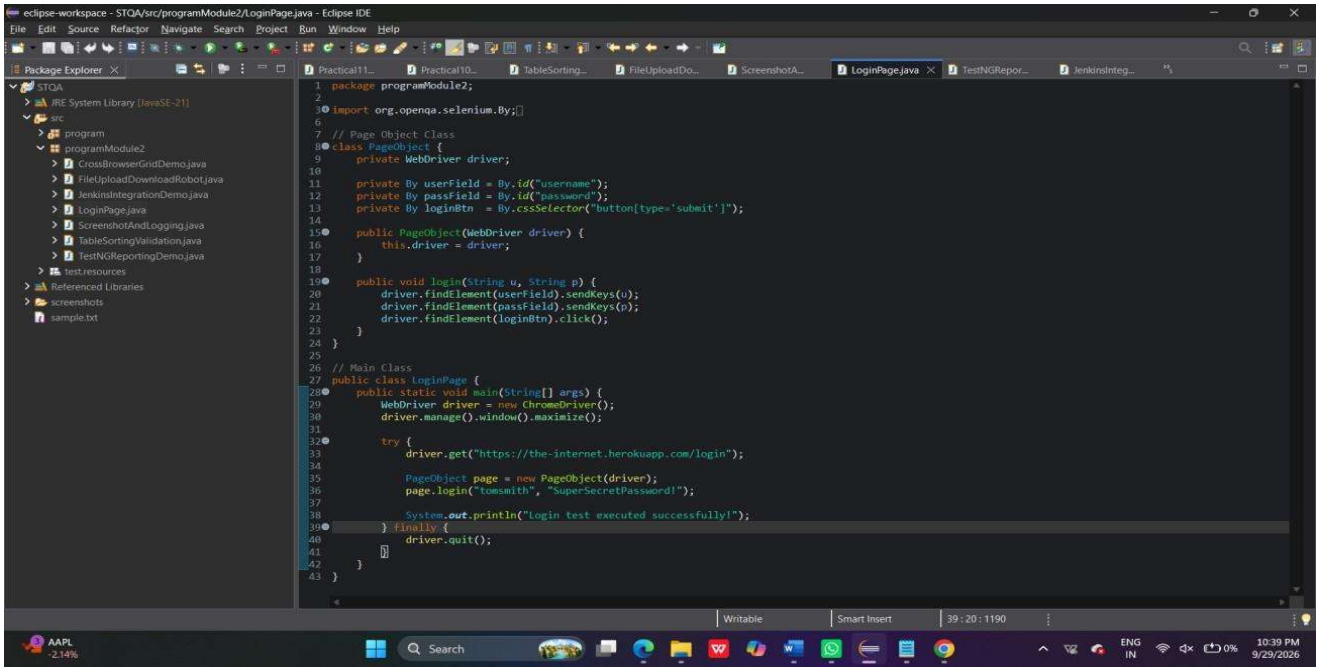


OUTPUT:



Practical No. 15: Page Object Model (POM) Framework Implementation

Aim: To design and implement an automation framework using Page Object Model (POM) with TestNG, creating reusable page classes and executing modular test cases




```java
package programModule2;

import org.openqa.selenium.By;

// Page Object Class
class PageObject {
    private WebDriver driver;

    private By userField = By.id("username");
    private By passField = By.id("password");
    private By loginBtn  = By.cssSelector("button[type='submit']");

    public PageObject(WebDriver driver) {
        this.driver = driver;
    }

    public void login(String u, String p) {
        driver.findElement(userField).sendKeys(u);
        driver.findElement(passField).sendKeys(p);
        driver.findElement(loginBtn).click();
    }
}

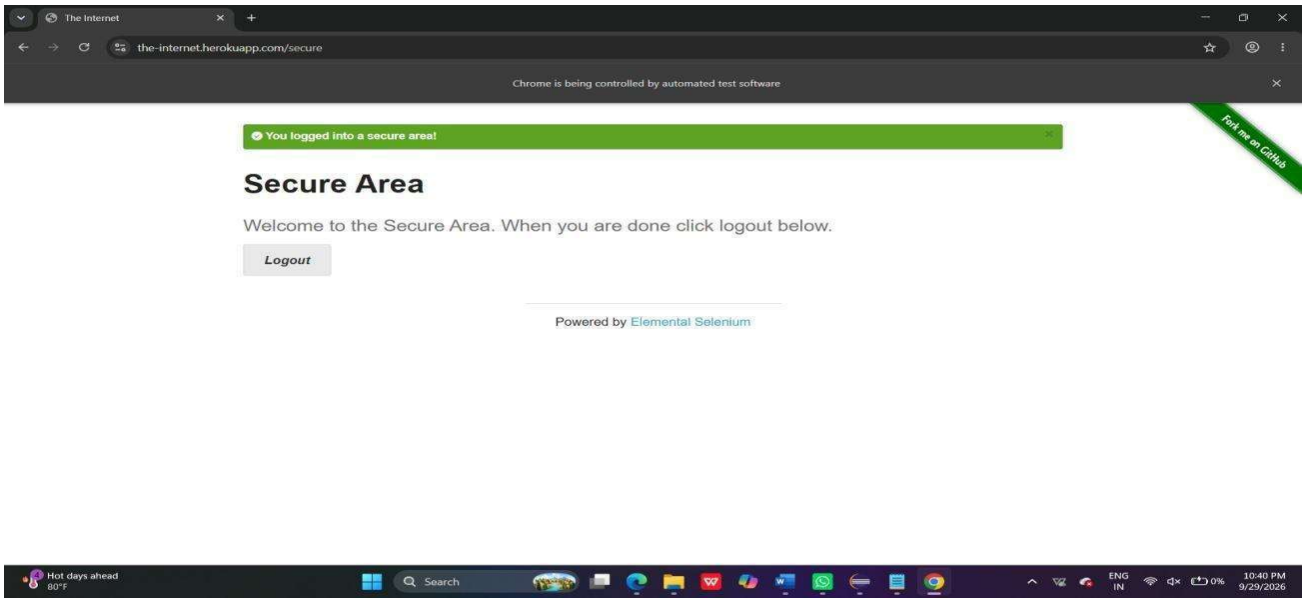
// Main Class
public class LoginPage {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();

        try {
            driver.get("https://the-internet.herokuapp.com/login");

            PageObject page = new PageObject(driver);
            page.login("tomsmith", "SuperSecretPassword!");

            System.out.println("Login test executed successfully!");
        } finally {
            driver.quit();
        }
    }
}
```

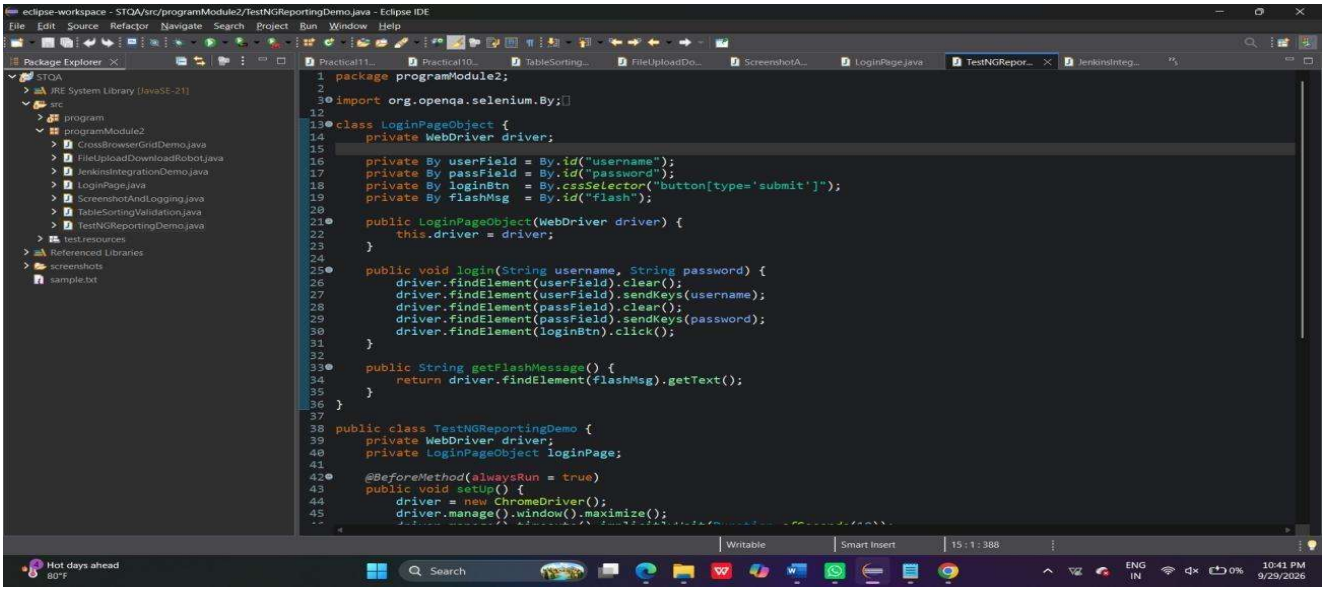
OUTPUT:

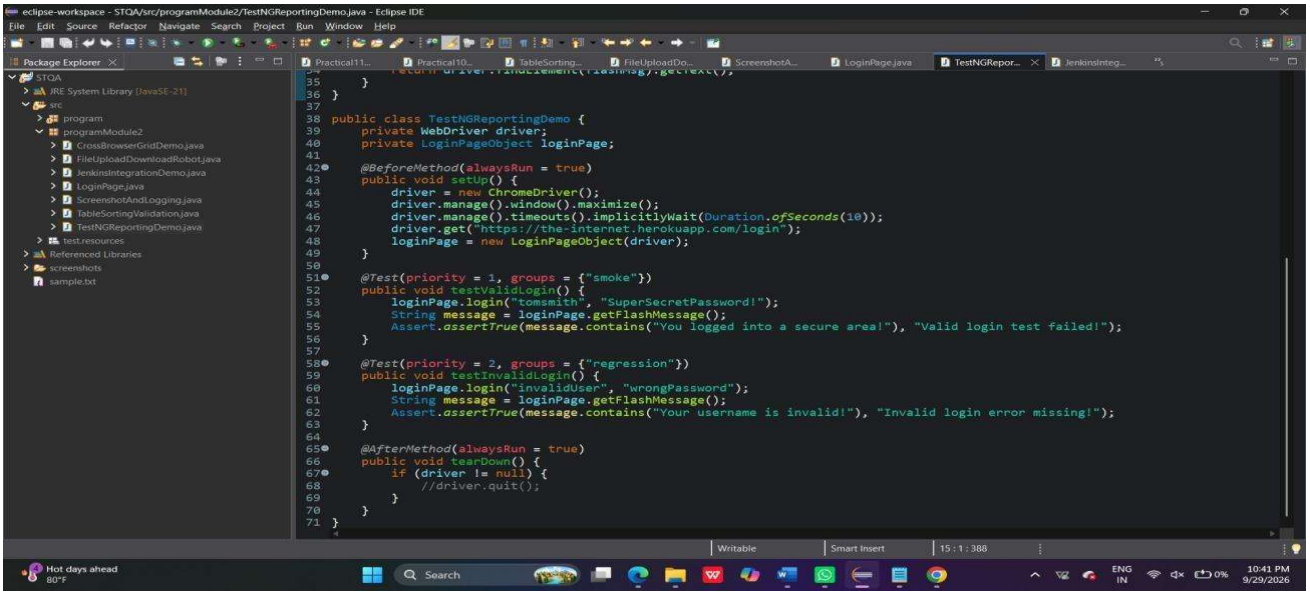



```
Login test executed successfully!
```

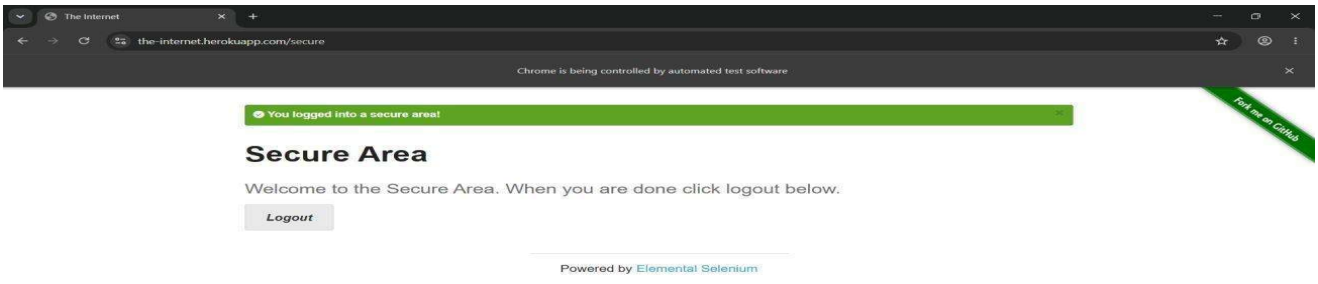
Practical No.17: TestNG Framework Implementation and Reporting

Aim: To create and execute automated test cases using TestNG annotations, group test cases, define priorities, and generate HTML test reports.





OUTPUT:



Practical No.18: Continuous Integration Using Jenkins

Aim: To configure Jenkins to execute Selenium automation scripts automatically, generate build reports, and analyze test execution history.

